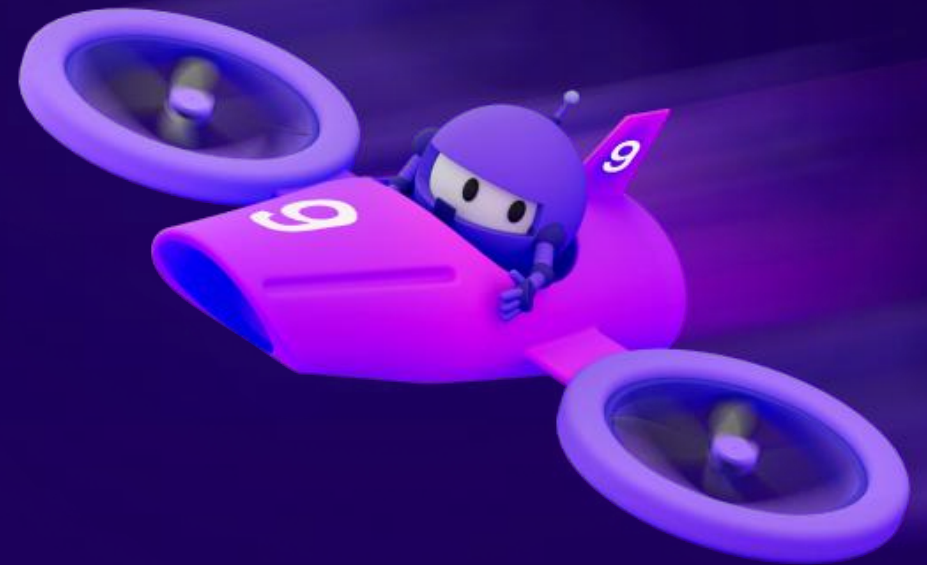


.NET을 이용한 벡터 그래픽스 기반 컴포넌트 구현

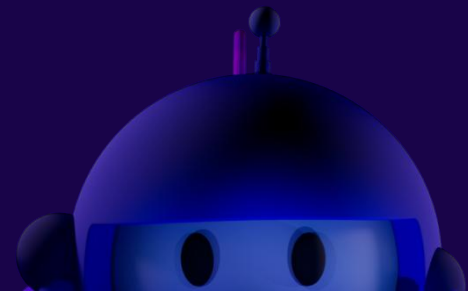
부제: 디지털 인디케이터 라이브러리

김준형



Agenda

1. 구현 배경
2. 벡터 그래픽스 기반 디지털 인디케이터
3. WPF, Avalonia UI, MAUI, Blazor 적용
4. 사용 예시 및 버그 개선
5. 향후 개발 방향



01

구현 배경

Blazor 기반 에어컨 원격 제어

- 더운 여름, 개인 작업실에 들어가기 전에 원격으로 미리 에어컨을 켜 놓을 목적으로 만듦
- 그런데 설정 온도 숫자 디자인이 마음에 안 듦

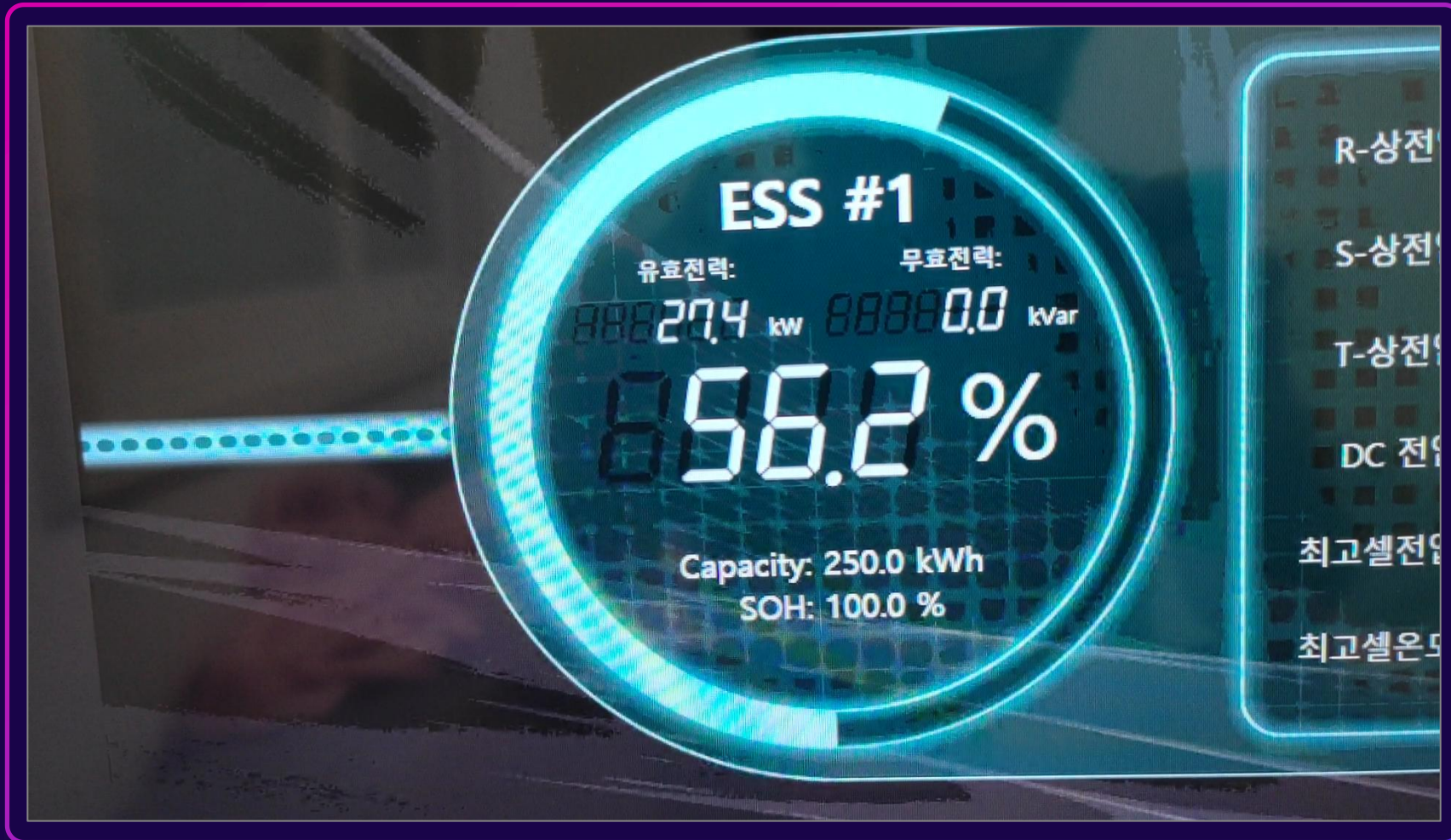


7-Segment Display

- ▶ 표시 장치의 일종으로 7개의 획으로 숫자나 문자를 나타낼 수 있으며, 전자 회로의 내부적인 수치를 보여 주는 데 자주 사용



기존 WPF 프로젝트 구현 예

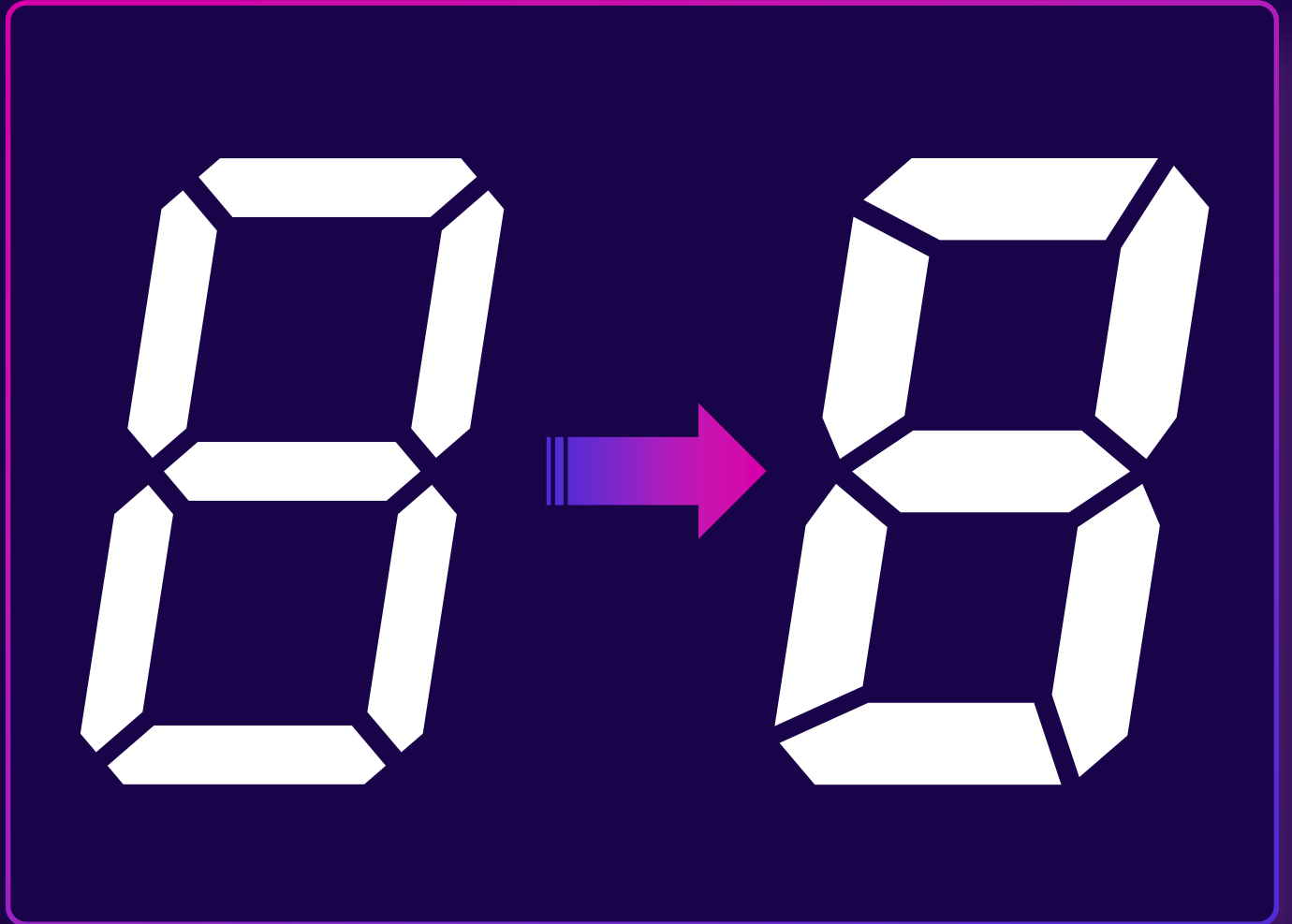


기존에 구현했던 벡터 그래픽 원본



조금 더 예쁘게 개선하고 싶다!

- 임팩트 있게 굵은 모양으로
 - 중간 틈을 더 자연스럽게
 - 숫자 1을 조금 더 크게
-
- 그러나... 그리기 귀찮다...
 - 또 다른 모양이 필요할 때도
또 다시 그려야 하나?



**“파라미터에 따라 디지털 숫자를
자동으로 생성하면 어떨까?”**

디지털 숫자를 만들어보는 김에...

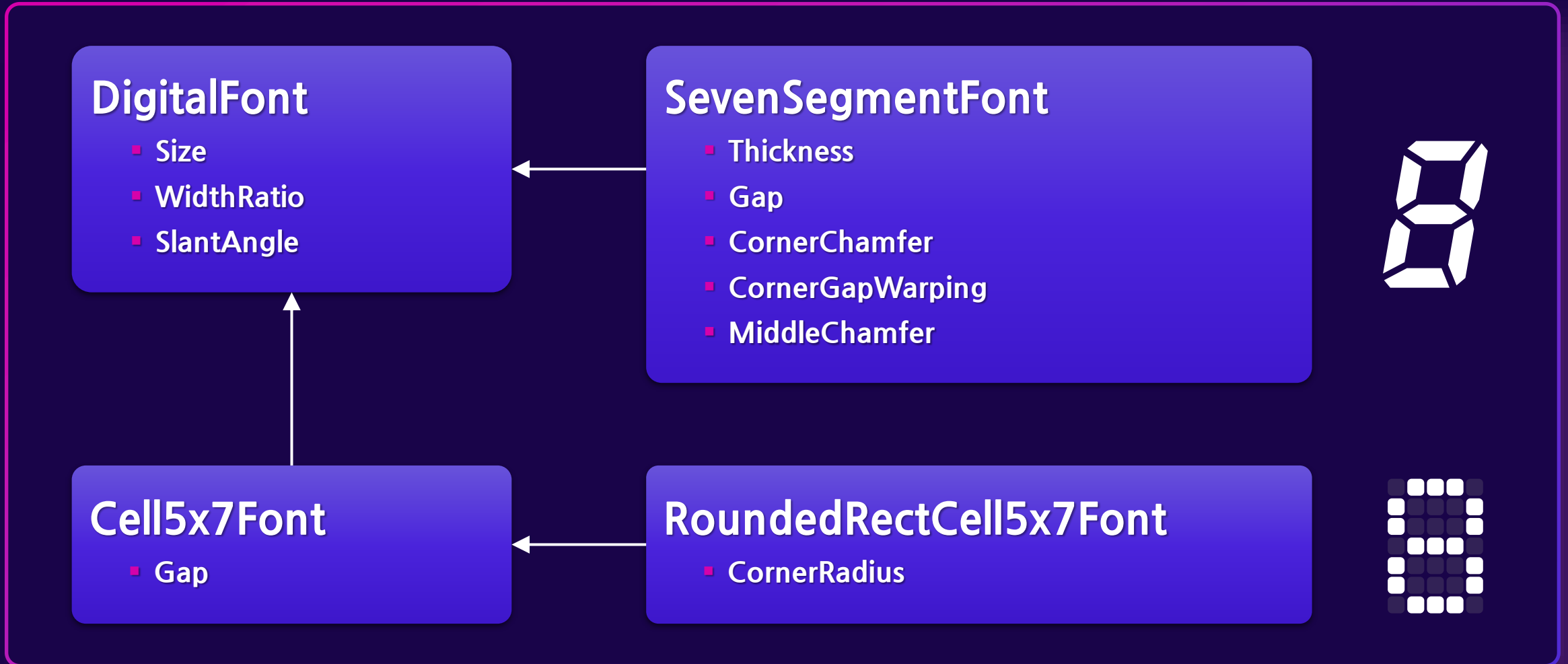
- 다른 디지털 폰트도 함께 만들어보자!
 - 기본 목표: 7-Segment 숫자
 - 추가 목표: 5 × 7 Dot matrix 문자
- 디지털 인디케이터 컴포넌트를 만들어보자!
 - 세그먼트 점멸 상태 구분 필요
 - 세그먼트 점멸 상태별 레이어 구현
- 다양한 플랫폼을 지원하도록 만들어보자!
 - WPF, Avalonia UI, MAUI, Blazor



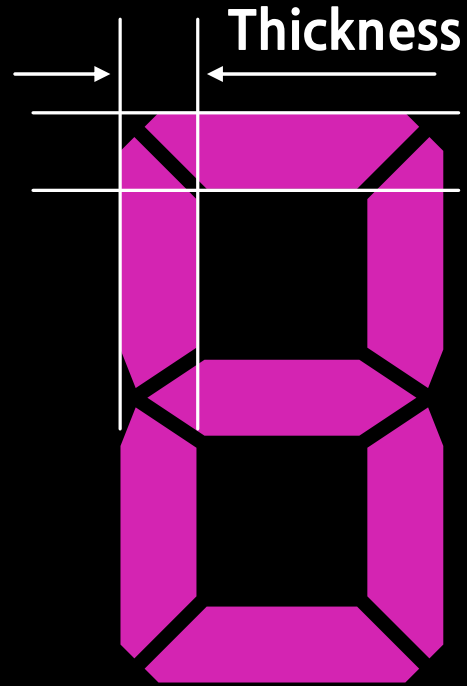
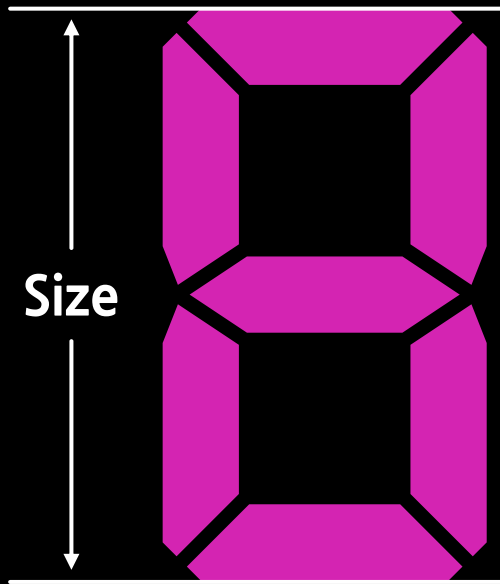
02

벡터 그래픽스 기반 디지털 인디케이터

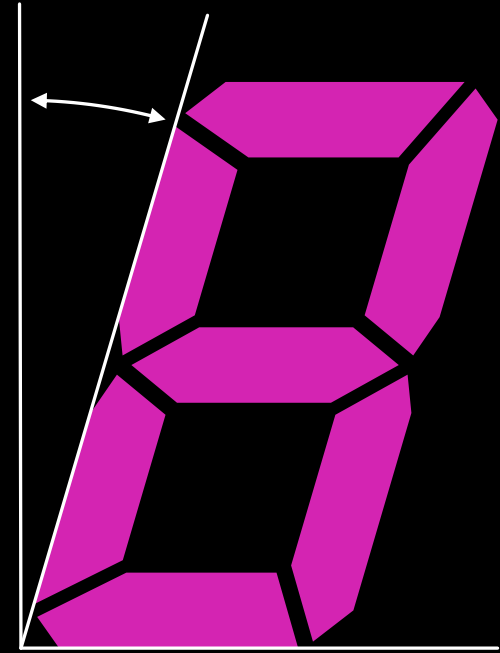
디지털 폰트 클래스 정의



폰트 크기, 세그먼트 굵기, 기울임 각 속성



Slant Angle



너비 비율 속성

Width Ratio

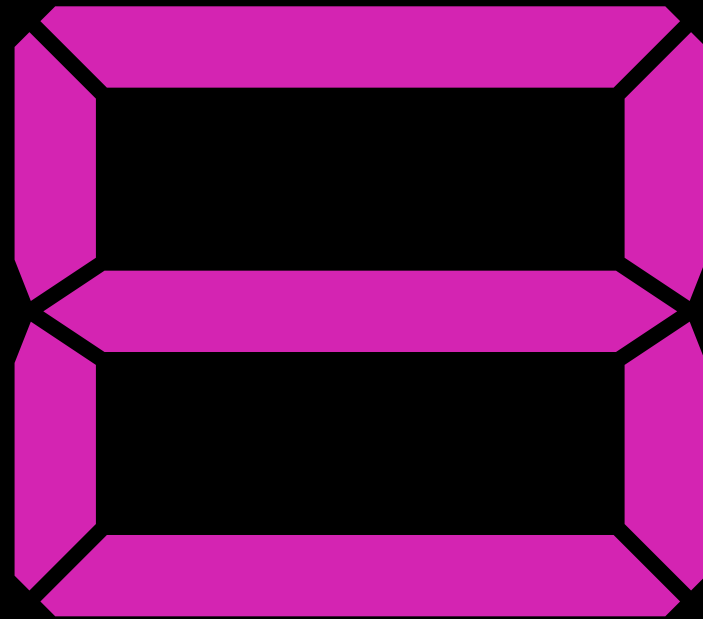
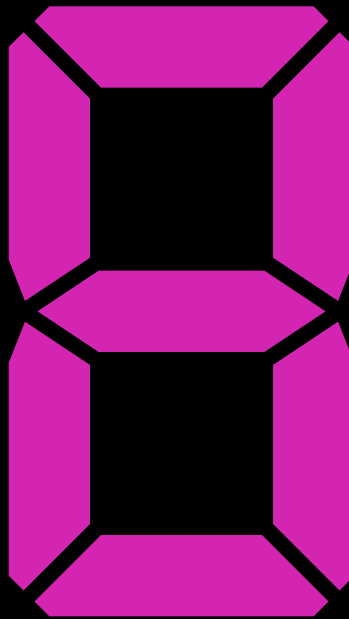
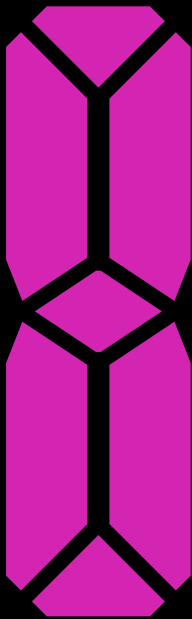
0.0

=

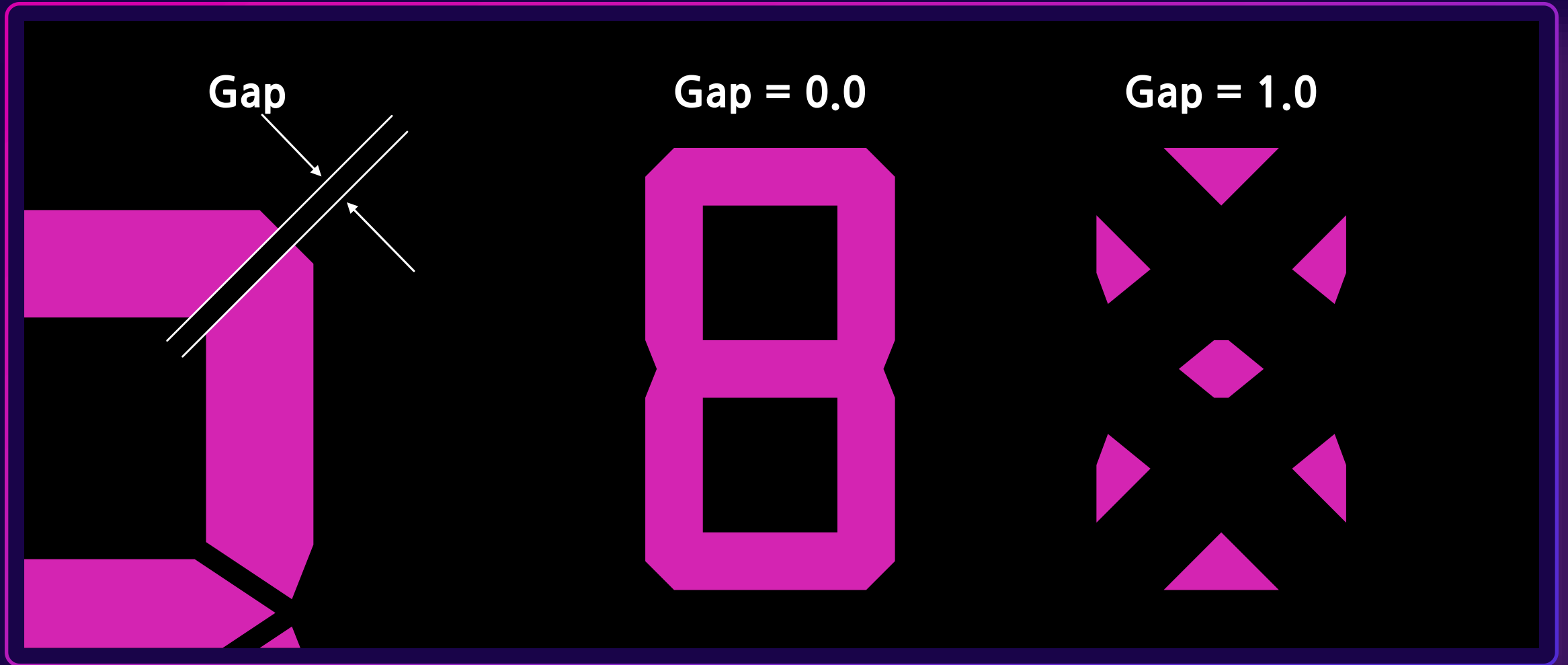
0.5

1.0

2.0

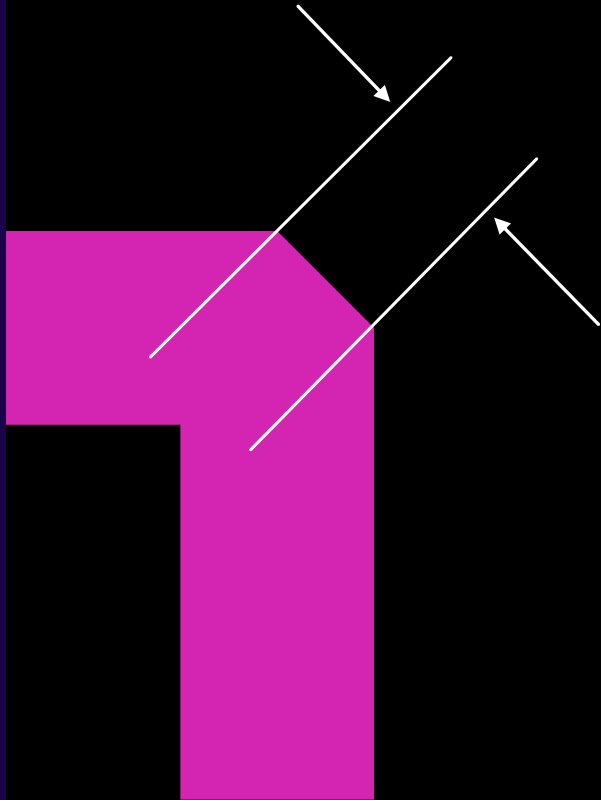


세그먼트 틈 간격 속성

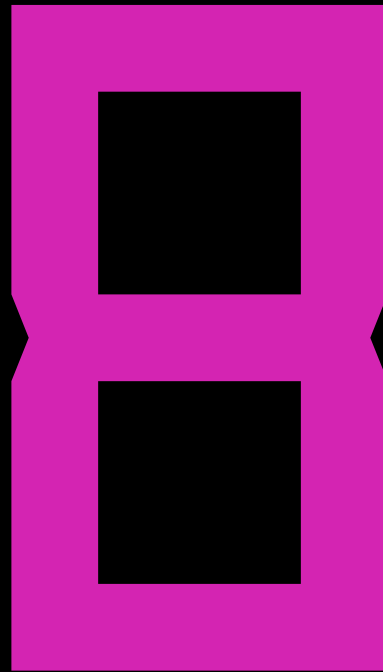


모퉁이 깎기 속성

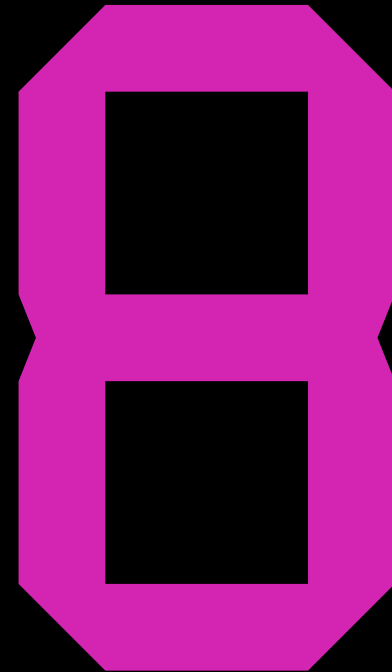
Corner Chamfer



Corner Chamfer = 0.0



Corner Chamfer = 1.0



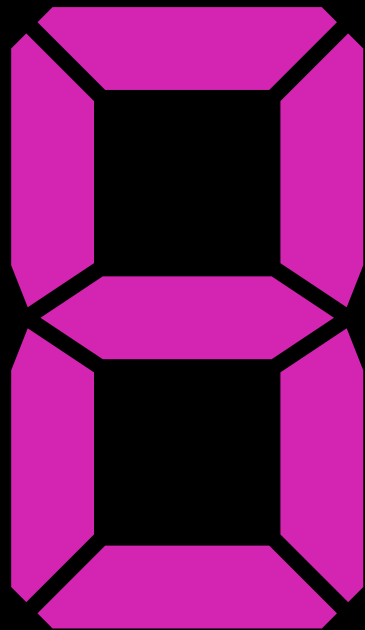
중간 연결점 깎기 속성



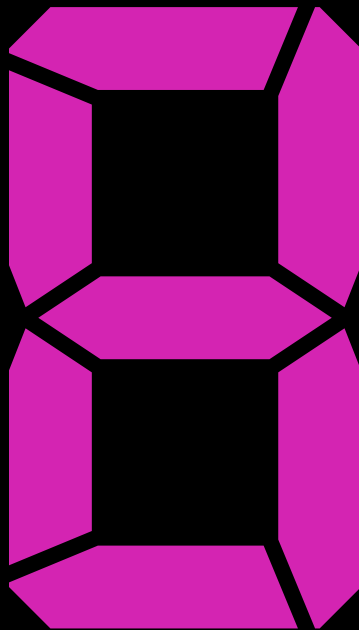
모퉁이 틈 뒤틀림 속성

Corner Gap Warping

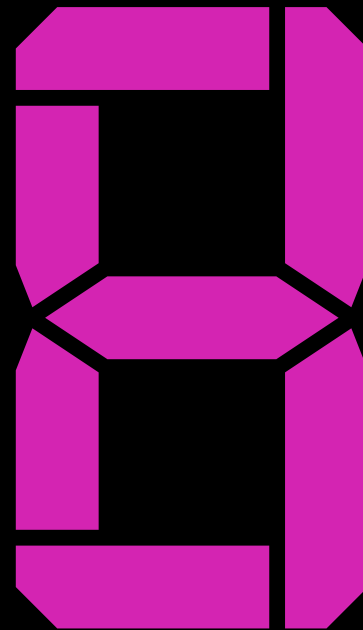
0.0



0.5

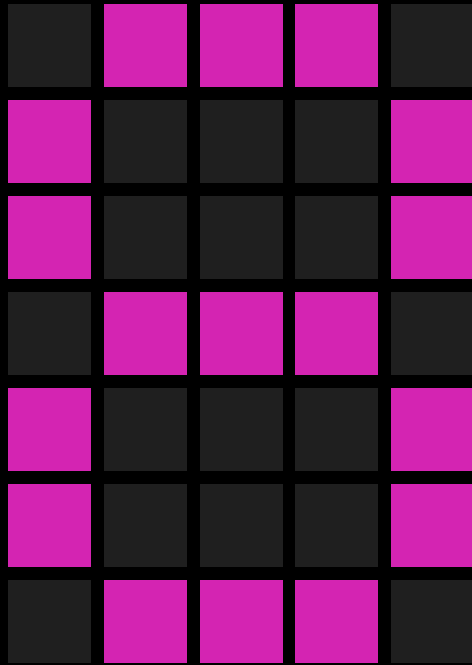


1.0

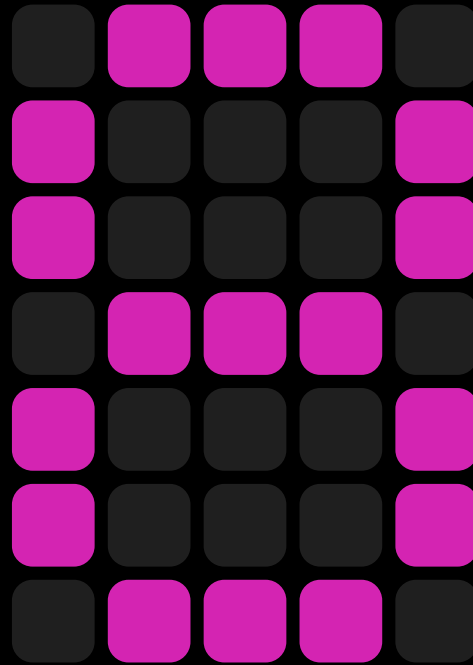


도트 모퉁이 반지름 속성

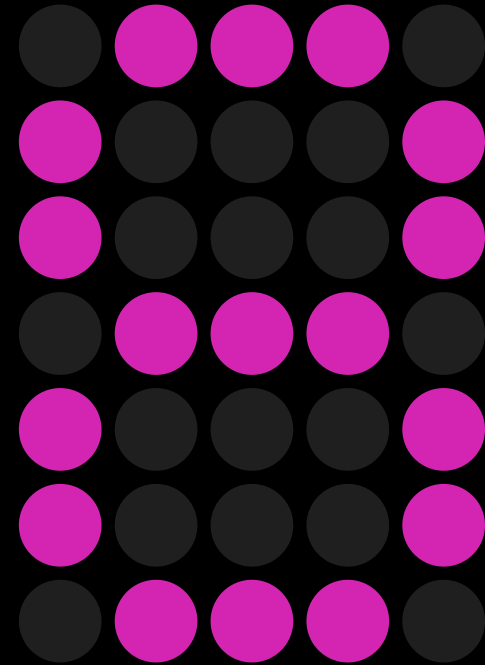
Corner Radius = 0.0



Corner Radius = 0.5

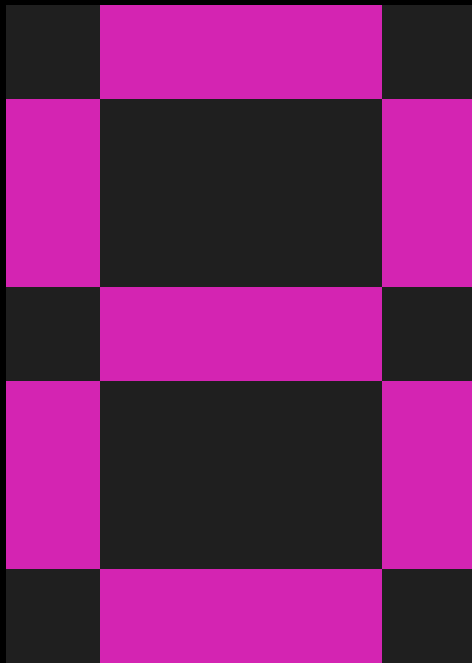


Corner Radius = 1.0

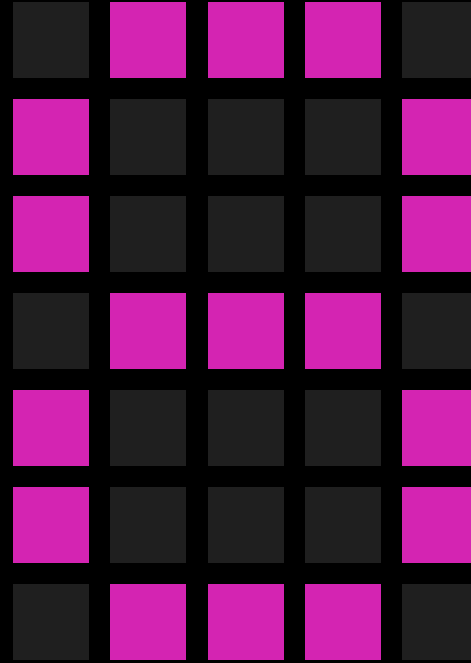


도트 매트릭스 간격 속성

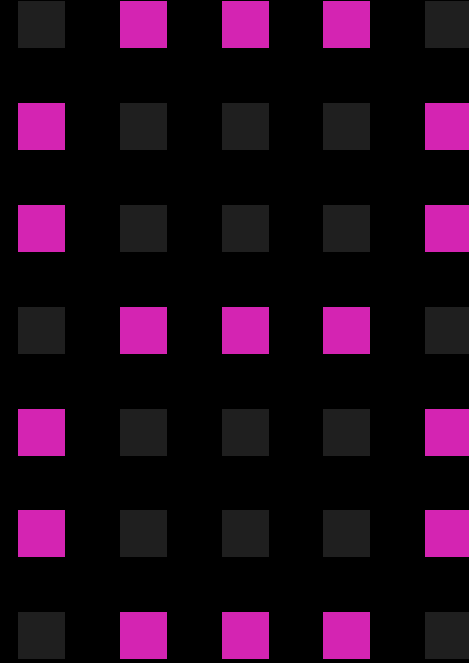
Gap = 0.0



Gap = 0.2



Gap = 0.5



Variations

123.45

123.45

123.45

123.45

123.45

123.45

123.45

123.45

123.45

Variations

HELLO, WORLD!

HELLO, WORLD!

HELLO, WORLD!

HELLO, WORLD!

파라미터에 따른 그리기 행위 관리

- 파라미터에 따른 벡터 그래픽을 여러 플랫폼에서 사용 가능하면 좋겠다!
 - 플랫폼 중립적 그리기 행위를 수행하기 위한 Context 구현 필요
- 그래픽을 위한 Context 예시
 - Win32 API의 Graphic Device Context
 - Windows Forms의 Graphics 클래스
 - WPF의 DrawingContext 클래스
- 그러면 파라미터에 따른 그리기 행위들은 어떻게 관리할 것인가?
 - Context를 이용한 그리기 행위를 Action으로 정의

IPartDrawingContext 인터페이스 정의

C#

```
// 파트 드로잉 컨텍스트 인터페이스. 파트의 모양을 패스 기반으로 그리기 위한 최소한의 메서드를 정의.
public interface IPartDrawingContext
{
    // 시작점을 지정하여 패스를 그리기를 시작합니다.
    void BeginPath(Point point);

    // 특정 지점을 향해 선분을 그립니다.
    void DrawLine(Point point);

    // 3차 베지어 곡선을 그립니다.
    void DrawCubicBezier(Point control1, Point control2, Point end);

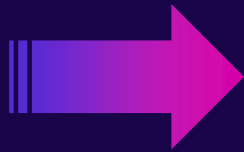
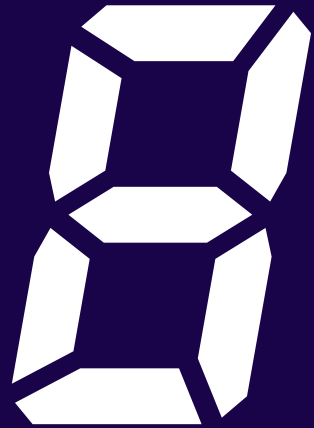
    // 2차 베지어 곡선을 그립니다.
    void DrawQuadraticBezier(Point control, Point end);

    // 패스를 닫습니다.
    void Close();

    // 파트를 그립니다.
    void DrawPart(in Part part);
}
```

그래픽 재사용을 위한 파트 관리

개별 세그먼트 드로잉



위치 변환

위치 및 크기 변환

Part 구조체 정의

C#

// 벡터 그래픽 기반 컴포넌트를 구성하는 파트, 재사용성을 위한

```
public readonly struct Part
```

```
{
```

```
    public Part(PartDrawing drawing) : this(drawing, Transform.Identity) { }
```

```
    public Part(PartDrawing drawing, Transform transform)
```

```
    {
```

```
        Drawing = drawing;
```

```
        Transform = transform;
```

```
    }
```

// 파트 드로잉, 파트를 그리기 위한 행위

```
public PartDrawing Drawing { get; }
```

// 변환

```
public Transform Transform { get; }
```

```
...
```

```
}
```

개별 세그먼트 드로잉



위치 변환



위치 및 크기 변환

PartDrawing 및 DelegateDrawing

C#

```
/// 파트를 그리기 위한 행위를 정의합니다.
public abstract class PartDrawing
{
    /// 드로잉 행위가 있는지 여부를 가져옵니다.
    public abstract bool IsEmpty { get; }

    /// 파트 드로잉 컨텍스트에 그리웁니다.
    protected abstract void OnDraw(IPartDrawingContext context);

    internal void DrawTo(IPartDrawingContext context) => OnDraw(context);
}

// 대리자를 이용한 파트 드로잉을 정의합니다.
public class DelegateDrawing : PartDrawing
{
    public DelegateDrawing(Action<IPartDrawingContext> drawingAction) => this.drawingAction = drawingAction;

    private readonly Action<IPartDrawingContext> drawingAction;

    /// 드로잉 행위가 있는지 여부를 가져옵니다.
    public override bool IsEmpty => drawingAction == null;

    protected override void OnDraw(IPartDrawingContext context) => drawingAction?.Invoke(context);
}
```

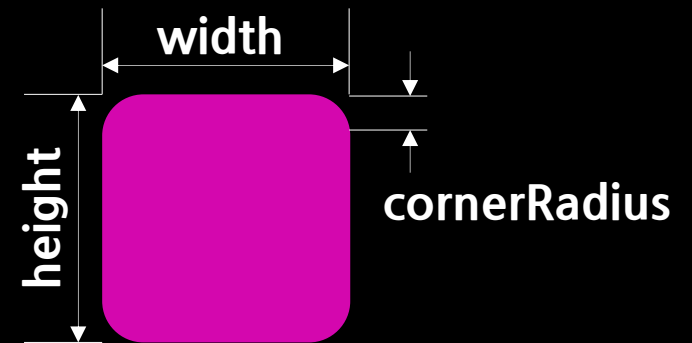
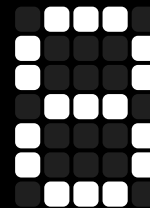


DelegateDrawing 기반 Dot 그리기 예시

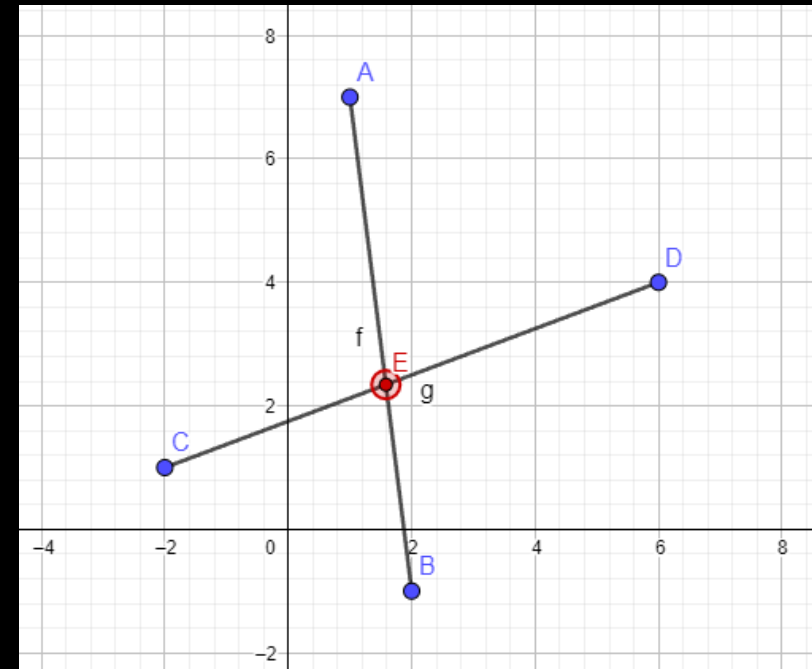
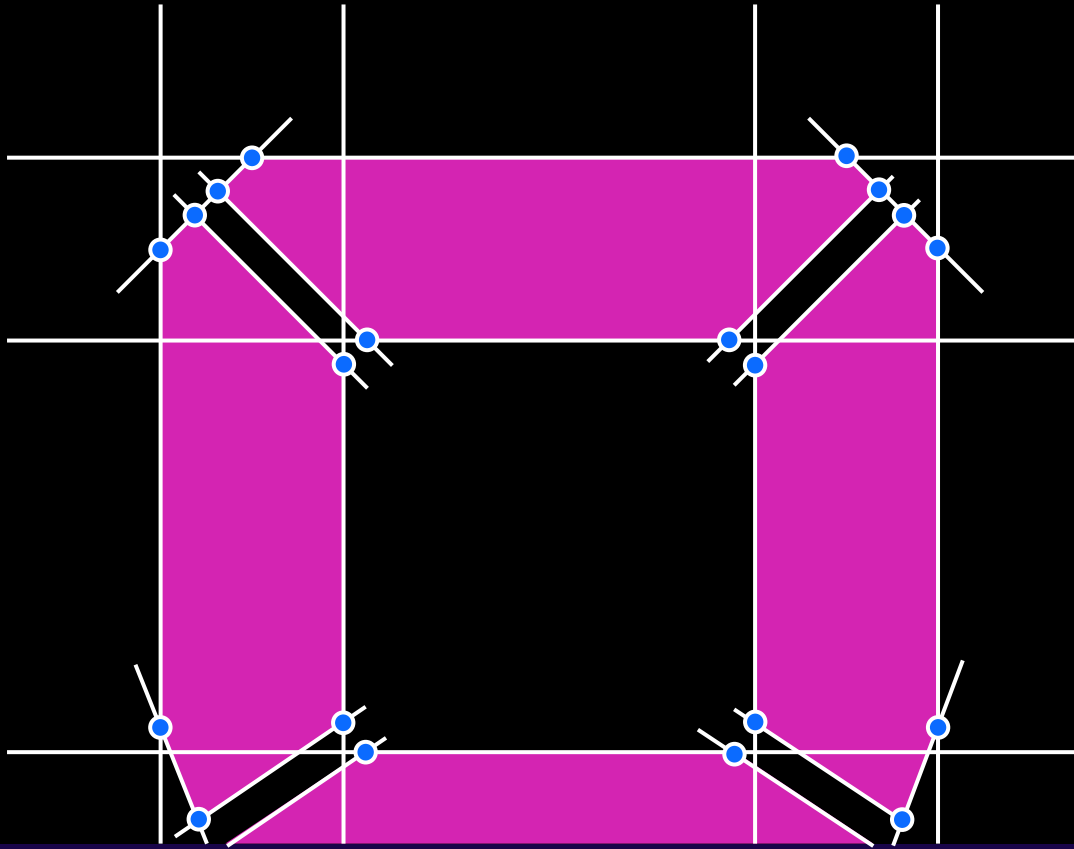
C#

```
var center1 = Math.Min(cornerRadius, width / 2);
var center2 = width - center1;
var middle1 = Math.Min(cornerRadius, height / 2);
var middle2 = height - middle1;
var cpOffsetX = center1 * arcBezierRatio;
var cpOffsetY = middle1 * arcBezierRatio;
var cpOffsetX1 = center1 - cpOffsetX;
var cpOffsetX2 = center2 + cpOffsetX;
var cpOffsetY1 = middle1 - cpOffsetY;
var cpOffsetY2 = middle2 + cpOffsetY;
var leftTop = new CubicBezier(new Point(0, middle1), new Point(0, cpOffsetY1), new Point(cpOffsetX1, 0), new Point(center1, 0));
var rightTop = new CubicBezier(new Point(center2, 0), new Point(cpOffsetX2, 0), new Point(width, cpOffsetY1), new Point(width, middle1));
var rightBottom = new CubicBezier(new Point(width, middle2), new Point(width, cpOffsetY2), new Point(cpOffsetX2, height), new Point(center2, height));
var leftBottom = new CubicBezier(new Point(center1, height), new Point(cpOffsetX1, height), new Point(0, cpOffsetY2), new Point(0, middle2));

return new DelegateDrawing(context =>
{
    context.BeginPath(leftTop.Start);
    context.DrawCubicBezier(leftTop.Control1, leftTop.Control2, leftTop.End);
    context.DrawLine(rightTop.Start);
    context.DrawCubicBezier(rightTop.Control1, rightTop.Control2, rightTop.End);
    context.DrawLine(rightBottom.Start);
    context.DrawCubicBezier(rightBottom.Control1, rightBottom.Control2, rightBottom.End);
    context.DrawLine(leftBottom.Start);
    context.DrawCubicBezier(leftBottom.Control1, leftBottom.Control2, leftBottom.End);
    context.Close();
});
```



7-Segment는 어떻게 그릴 것인가?



선분의 교점 계산

선분 정의 - Line 구조체

C#

```
public struct Line
{
    public Line(in double startX, in double startY, in double endX, in double endY)
    {
        start = new Point(startX, startY);
        end = new Point(endX, endY);
    }

    private Point start;
    private Point end;

    public Point Start { get => start; set => start = value; }
    public Point End { get => end; set => end = value; }
    public double DeltaX => end.x - start.x;
    public double DeltaY => end.y - start.y;
    public double Direction => Math.Atan2(DeltaY, DeltaX);
    public double Length => Math.Sqrt(Math.Pow(DeltaX, 2) + Math.Pow(DeltaY, 2));

    public bool TryGetCrossPoint(in Line line, out Point crossPoint, bool checkIntersection = true) { ... }
    public bool Intersection(in Line line) { ... }
}
```


선분 교점 계산 메서드

C#

```
public bool TryGetCrossPoint(in Line line, out Point crossPoint, bool checkIntersection = true)
{
    if (!checkIntersection || Intersection(line))
    {
        var dx1 = this.DeltaX;
        var dy1 = this.DeltaY;
        var dx2 = line.DeltaX;
        var dy2 = line.DeltaY;
        var d = dx1 * dy2 - dy1 * dx2;
        if (d != 0)
        {
            var a = this.start.y * this.end.x - this.start.x * this.end.y;
            var b = line.start.y * line.end.x - line.start.x * line.end.y;
            crossPoint = new Point((a * dx2 - dx1 * b) / d, (a * dy2 - dy1 * b) / d);
            return true;
        }
    }
    crossPoint = default;
    return false;
}
```

선분 교차 여부 확인 메서드

C#

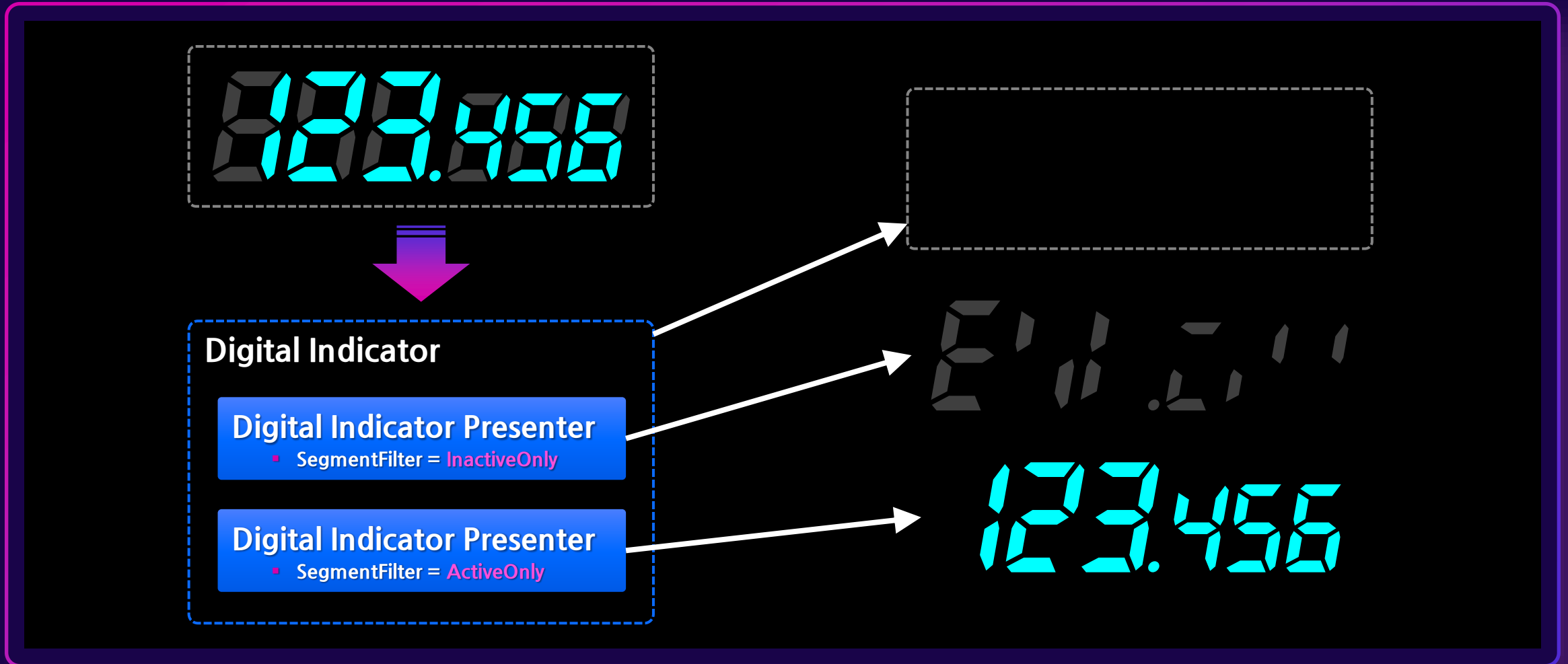
```
public bool Intersection(in Line line)
{
    int Ccw(in Point point1, in Point point2, in Point point3)
    {
        var crossProduct = (point2.x - point1.x) * (point3.y - point1.y) - (point3.x - point1.x) * (point2.y - point1.y);
        return crossProduct > 0 ? 1 : crossProduct < 0 ? -1 : 0;
    }

    bool Comparator(in Point point1, in Point point2) => point1.x == point2.x ? point1.y <= point2.y : point1.x <= point2.x;

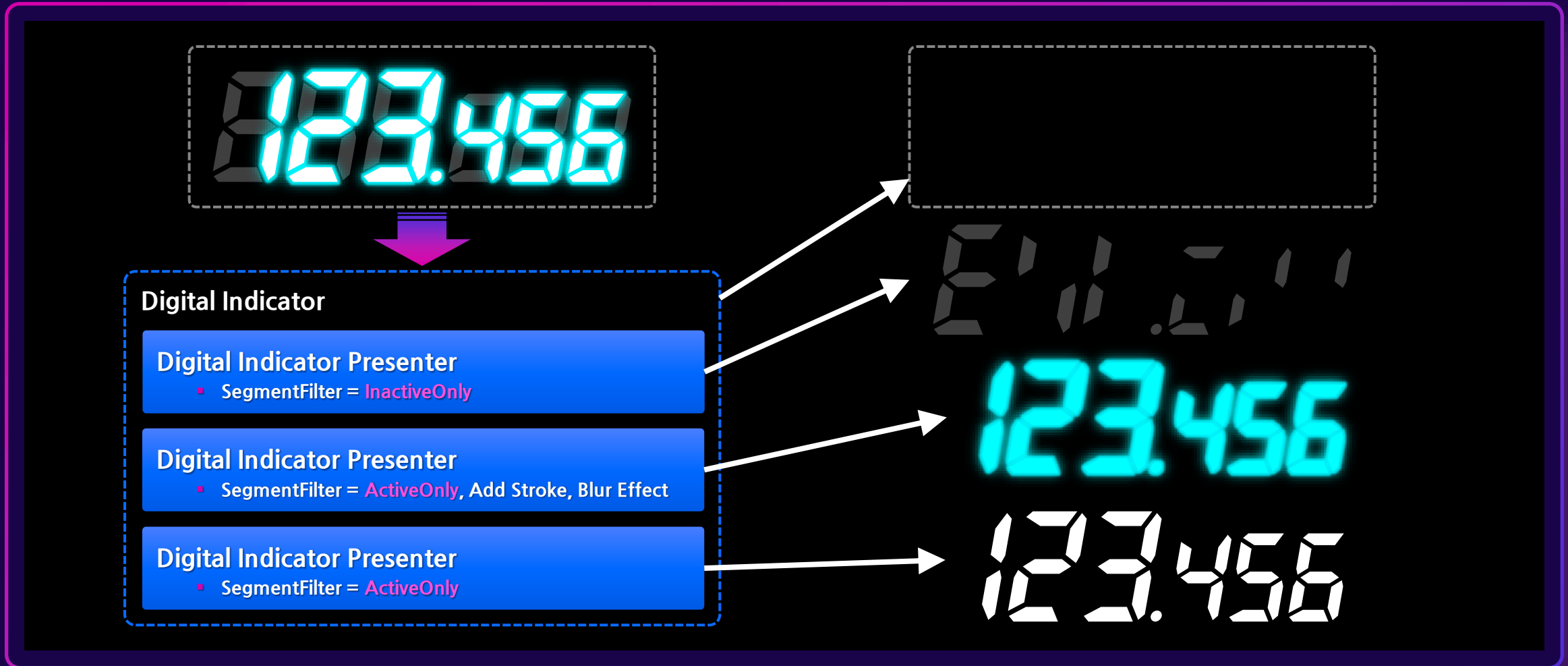
    int l1_l2 = Ccw(this.start, this.end, line.start) * Ccw(this.start, this.end, line.end);
    int l2_l1 = Ccw(line.start, line.end, this.start) * Ccw(line.start, line.end, this.end);

    if (l1_l2 == 0 && l2_l1 == 0)
    {
        if (Comparator(end, start))
            return Comparator(line.start, this.start) && Comparator(this.end, line.end);
        else if (Comparator(line.end, line.start))
            return Comparator(line.end, this.end) && Comparator(this.start, line.start);
        else
            return Comparator(line.start, this.end) && Comparator(this.start, line.end);
    }
    else
        return l1_l2 <= 0 && l2_l1 <= 0;
}
```

다중 레이어 기반 디지털 인디케이터

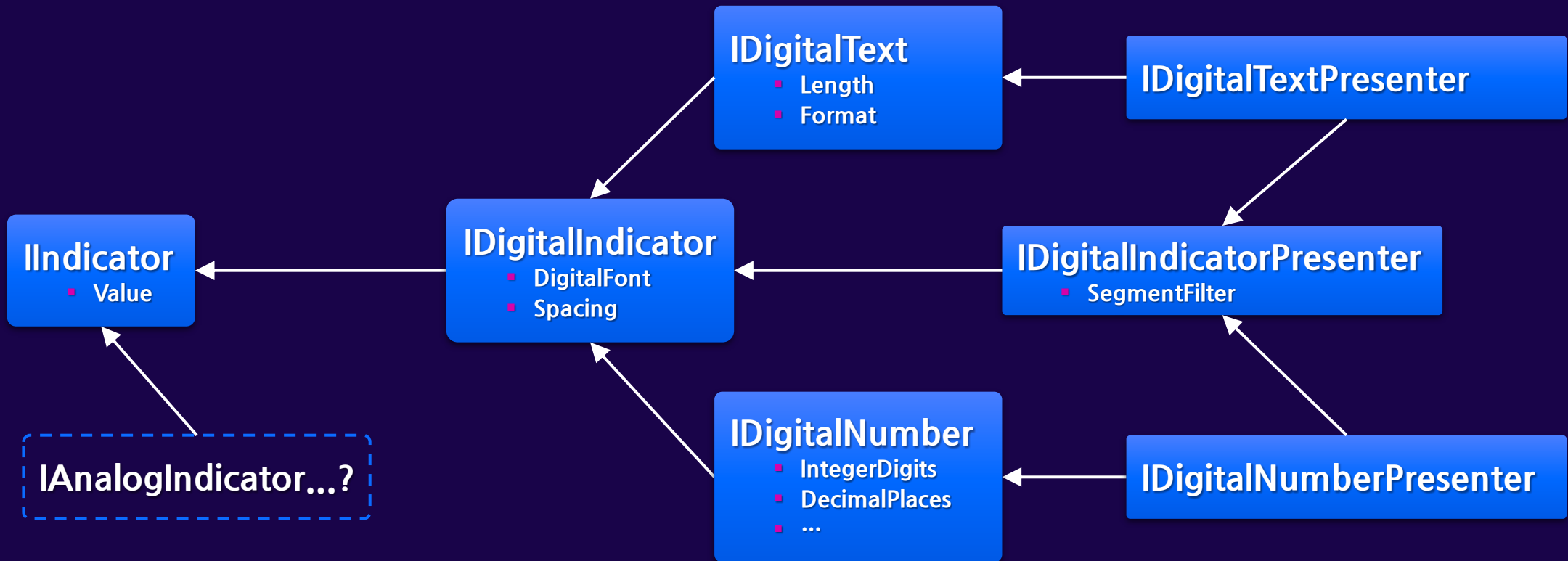


다중 레이어 구조를 이용한 Glow 효과



디지털 인디케이터 인터페이스 정의

Common Interfaces in .NET Standard 2.0



인디케이터 속성을 이용한 파트 목록 생성

C#

```
public static class DigitalTextExtentions
{
    public static IEnumerable<Part> CreateParts(this IDigitalText indicator, DigitalSegmentFilter segmentFilter)
    {
        var font = indicator?.DigitalFont;
        if (font == null) yield break;

        var length = indicator.Length;

        var text = (string.IsNullOrEmpty(indicator.Format)
            ? (indicator.Value?.ToString() ?? string.Empty)
            : string.Format($"{{0:{{indicator.Format}}}}", indicator.Value)).PadRight(length, ' ');

        var strokeThickness = 0;
        var charWidth = font.Width;
        var spacing = charWidth * indicator.Spacing.ToValidValue();
        var halfStrokeThickness = strokeThickness / 2;
        var pitch = charWidth + strokeThickness + spacing;

        for (int i = 0; i < length; i++)
            foreach (var part in font.GetCharacterSegments(text[i], segmentFilter))
                yield return part * Transform.CreateTranslation(halfStrokeThickness + pitch * i + spacing, spacing + halfStrokeThickness);
    }
}
```

03

**WPF, Avalonia UI,
MAUI, Blazor 적용**

플랫폼 중립적 설계

- 파라미터 기반 좌표 계산 및 도형 생성은 .NET Standard 2.0 기반으로 구현
 - 아직도 .NET Framework를 버리지 못하는 지방 중소기업...
- 플랫폼별 Drawing Context 렌더러
 - WPF와 Avalonia UI 의 StreamGeometryContext
 - MAUI의 PathF
 - Blazor의 Inline SVG

PartDrawingContext 클래스 정의

C#

```
using VagabondK.Indicators.GeometryUtil;
// TRenderer: 컨텍스트를 통한 패스 그리기를 실제로 수행할 렌더러의 형식입니다.
// TRendererPoint: 렌더러에서 사용할 포인트 형식입니다.
public abstract class PartDrawingContext<TRenderer, TRendererPoint> : IPartDrawingContext
{
    // 컨텍스트를 통한 패스 그리기를 실제로 수행할 렌더러를 가져오거나 설정합니다.
    public TRenderer Renderer { get; set; }

    public void BeginPath(Point startPoint) { ... }
    public void DrawLine(Point endPoint) { ... }
    public void DrawCubicBezier(Point control1, Point control2, Point end) { ... }
    public void DrawQuadraticBezier(Point control, Point end) { ... }
    public void Close() { ... }
    public virtual void DrawPart(in Part part) { ... }

    protected abstract void OnBeginPath(in TRendererPoint startPoint);
    protected abstract void OnDrawLine(in TRendererPoint endPoint);
    protected abstract void OnDrawCubicBezier(in TRendererPoint controlPoint1, in TRendererPoint controlPoint2, in TRendererPoint endPoint);
    protected abstract void OnDrawQuadraticBezier(in TRendererPoint controlPoint, in TRendererPoint endPoint);
    protected abstract void OnClosePath();
    protected abstract TRendererPoint OnConvertToRendererPoint(in Point point);
}
```

PartDrawingContext 상속 및 구현



C#

```
using System.Windows;
using System.Windows.Media;

public class PartDrawingContext : PartDrawingContext<StreamGeometryContext, Point>
{
    protected override void OnBeginPath(in Point startPoint) => Renderer.BeginFigure(startPoint, true, true);

    protected override void OnDrawLine(in Point endPoint) => Renderer.LineTo(endPoint, true, true);

    protected override void OnDrawCubicBezier(in Point controlPoint1, in Point controlPoint2, in Point endPoint)
        => Renderer.BezierTo(controlPoint1, controlPoint2, endPoint, true, true);

    protected override void OnDrawQuadraticBezier(in Point controlPoint, in Point endPoint)
        => Renderer.QuadraticBezierTo(controlPoint, endPoint, true, true);

    protected override void OnClosePath() { }

    protected override Point OnConvertToRendererPoint(in GeometryUtil.Point point) => new Point(point.X, point.Y);
}
```

PartDrawingContext 상속 및 구현



C#

```
using Avalonia;
using Avalonia.Media;

public class PartDrawingContext : PartDrawingContext<StreamGeometryContext, Point>
{
    protected override void OnBeginPath(in Point startPoint) => Renderer.BeginFigure(startPoint, true);

    protected override void OnDrawLine(in Point endPoint) => Renderer.LineTo(endPoint);

    protected override void OnDrawCubicBezier(in Point controlPoint1, in Point controlPoint2, in Point endPoint)
        => Renderer.CubicBezierTo(controlPoint1, controlPoint2, endPoint);

    protected override void OnDrawQuadraticBezier(in Point controlPoint, in Point endPoint)
        => Renderer.QuadraticBezierTo(controlPoint, endPoint);

    protected override void OnClosePath() => Renderer.EndFigure(true);

    protected override Point OnConvertToRendererPoint(in GeometryUtil.Point point) => new Point(point.X, point.Y);
}
```

PartDrawingContext 상속 및 구현



C#

```
using Microsoft.Maui.Graphics;

public class PartDrawingContext : PartDrawingContext<PathF, Point>
{
    protected override void OnBeginPath(in Point startPoint) => Renderer.MoveTo(startPoint);

    protected override void OnDrawLine(in Point endPoint) => Renderer.LineTo(endPoint);

    protected override void OnDrawCubicBezier(in Point controlPoint1, in Point controlPoint2, in Point endPoint)
        => Renderer.CurveTo(controlPoint1, controlPoint2, endPoint);

    protected override void OnDrawQuadraticBezier(in Point controlPoint, in Point endPoint)
        => Renderer.QuadTo(controlPoint, endPoint);

    protected override void OnClosePath() => Renderer.Close();

    protected override Point OnConvertToRendererPoint(in GeometryUtil.Point point) => new(point.X, point.Y);
}
```

PartDrawingContext 상속 및 구현



C#

```
public class SvgPathDrawingContext : PartDrawingContext<StringBuilder>
{
    protected override void OnBeginPath(in Point startPoint) => Renderer.Append('M').AppendPoint(startPoint);

    protected override void OnDrawLine(in Point endPoint) => Renderer.Append('L').AppendPoint(endPoint);

    protected override void OnDrawCubicBezier(in Point controlPoint1, in Point controlPoint2, in Point endPoint)
        => Renderer.Append('C').AppendPoint(controlPoint1).AppendPoint(controlPoint2).AppendPoint(endPoint);

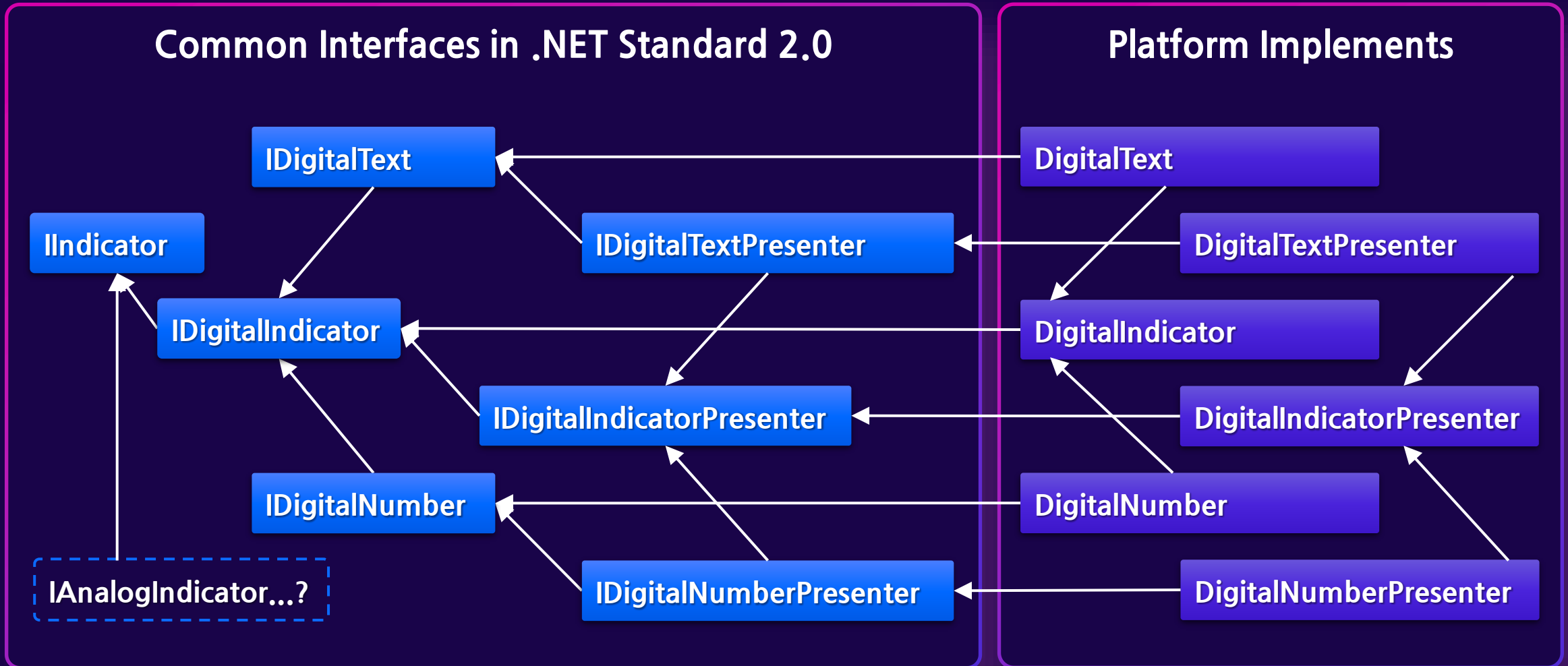
    protected override void OnDrawQuadraticBezier(in Point controlPoint, in Point endPoint)
        => Renderer.Append('Q').AppendPoint(controlPoint).AppendPoint(endPoint);

    protected override void OnClosePath() => Renderer.Append('Z');
}

static class StringBuilderExtensions
{
    public static StringBuilder AppendPoint(this StringBuilder stringBuilder, in Point point)
        => stringBuilder.Append(' ').Append(point.X).Append(' ').Append(point.Y);
}
```

```
<path d="M 0 3.1428571428571432C 0 1.4071050719603633 1.4071050719603633 0 3.1428571428571432 0L 9.42857142857143 0C 11.16432349946821 0 12.571428571428573 1.4071050719603633 12.571428571428573 3.1428571428571432L 12.571428571428573 9.428571428571427C 12.571428571428573 11.164323499468207 11.16432349946821 12.571428571428571 9.42857142857143 12.571428571428571L 3.1428571428571432 12.571428571428571C 1.4071050719603633 12.571428571428571 0 11.164323499468207 0 9.428571428571427Z"></path>
```

플랫폼 레벨 디지털 인디케이터 구현



플랫폼별 프로젝트 파일 목록



```
└─ VagabondK.Indicators.Windows
  └─ 종속성
  └─ Themes
    └─ C# AssemblyInfo.cs
    └─ C# DigitalIndicator.cs
    └─ C# DigitalIndicatorPresenter.cs
    └─ C# DigitalNumber.cs
    └─ C# DigitalNumberPresenter.cs
    └─ C# DigitalText.cs
    └─ C# DigitalTextPresenter.cs
    └─ nugeticon.png
    └─ C# PartDrawingContext.cs
```

WPF



```
└─ VagabondK.Indicators.Avalonia
  └─ 종속성
    └─ C# AssemblyInfo.cs
    └─ C# DefaultTemplatedControl.cs
    └─ C# DigitalIndicator.cs
    └─ C# DigitalIndicatorPresenter.cs
    └─ DigitalNumber.axaml
    └─ C# DigitalNumberPresenter.cs
    └─ DigitalText.axaml
    └─ C# DigitalTextPresenter.cs
    └─ nugeticon.png
    └─ C# PartDrawingContext.cs
    └─ C# TemplatedControlExtensions.cs
```

Avalonia UI



```
└─ VagabondK.Indicators.Maui
  └─ 종속성
    └─ C# AssemblyInfo.cs
    └─ C# DigitalIndicator.cs
    └─ C# DigitalIndicatorPresenter.cs
    └─ DigitalNumber.xaml
    └─ C# DigitalNumberPresenter.cs
    └─ DigitalText.xaml
    └─ C# DigitalTextPresenter.cs
    └─ C# MauiAppBuilderExtensions.cs
    └─ nugeticon.png
    └─ C# PartDrawingContext.cs
```

MAUI



```
└─ VagabondK.Indicators.Razor
  └─ wwwroot
    └─ 종속성
      └─ C# DigitalIndicator.cs
      └─ DigitalIndicatorPresenter.razor
      └─ DigitalNumber.razor
      └─ C# DigitalNumberPresenter.cs
      └─ DigitalText.razor
      └─ C# DigitalTextPresenter.cs
      └─ nugeticon.png
      └─ SharedDigitalFontSegments.razor
```

Blazor

DigitalIndicator 클래스



API 레퍼런스 / 네임스페이스 / VagabondK.Indicators.Windows / 클래스 / DigitalIndicator / 필드 / 속성

DigitalIndicator 클래스

값을 여러 세그먼트로 구성된 디지털 문자들로 표시하는 인디케이터입니다.

상속

```
object
├── DispatcherObject
│   ├── DependencyObject
│   │   ├── Visual
│   │   │   ├── UIElement
│   │   │   │   ├── FrameworkElement
│   │   │   │   │   ├── Control
│   │   │   │   │   │   ├── DigitalIndicator
│   │   │   │   │   │   │   ├── DigitalNumber
│   │   │   │   │   │   │   └── DigitalText
```

구현

ISupportInitialize

IDigitalIndicator

IIndicator

네임스페이스: VagabondK.Indicators.Windows

어셈블리: VagabondK.Indicators.Windows.dll

속성

이름	설명
Active	활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
ActiveStroke	활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
Inactive	비활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
InactiveStroke	비활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
StrokeDashArray	윤곽선의 대시 및 간격 컬렉션을 가져오거나 설정합니다.
StrokeDashCap	윤곽선의 각 대시 끝을 그리는 방법을 지정하는 값을 가져오거나 설정합니다.
StrokeDashOffset	대시 시퀀스와 윤곽선 시작 위치 사이의 간격을 가져오거나 설정합니다.
StrokeEndLineCap	윤곽선의 끝 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeLineJoin	윤곽선의 꼭짓점에 사용되는 이음새의 형식을 가져오거나 설정합니다.
StrokeMiterLimit	윤곽선 두께의 절반에 대한 마이터 길이의 비율 제한을 가져오거나 설정합니다.
StrokeStartLineCap	윤곽선의 시작 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeThickness	세그먼트 윤곽선 두께를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalIndicatorPresenter 클래스



API 레퍼런스 / 네임스페이스 / VagabondK.Indicators.Windows / 클래스 / DigitalIndicatorPresenter / 생성:

DigitalIndicatorPresenter 클래스

표시할 세그먼트에 대한 필터에 따라 디지털 문자들을 표시합니다.

상속

↳ object
↳ DispatcherObject
↳ DependencyObject
↳ Visual
↳ UIElement
↳ FrameworkElement
↳ DigitalIndicatorPresenter
↳ DigitalNumberPresenter
↳ DigitalTextPresenter

구현

ISupportInitialize
IDigitalIndicatorPresenter
IDigitalIndicator
IIndicator

네임스페이스: VagabondK.Indicators.Windows

어셈블리: VagabondK.Indicators.Windows.dll

속성

이름	설명
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
Fill	세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
SegmentFilter	표시할 세그먼트에 대한 필터를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
Stroke	세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
StrokeDashArray	윤곽선의 대시 및 간격 컬렉션을 가져오거나 설정합니다.
StrokeDashCap	윤곽선의 각 대시 끝을 그리는 방법을 지정하는 값을 가져오거나 설정합니다.
StrokeDashOffset	대시 시퀀스와 윤곽선 시작 위치 사이의 간격을 가져오거나 설정합니다.
StrokeEndLineCap	윤곽선의 끝 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeLineJoin	윤곽선의 꼭짓점에 사용되는 이음새의 형식을 가져오거나 설정합니다.
StrokeMiterLimit	윤곽선 두께의 절반에 대한 마이터 길이의 비율 제한을 가져오거나 설정합니다.
StrokeStartLineCap	윤곽선의 시작 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeThickness	세그먼트 윤곽선 두께를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalNumber 템플릿 정의



```
<ControlTemplate TargetType="{x:Type vki:DigitalNumber}">
  <Border Background="{TemplateBinding Background}"
    BorderBrush="{TemplateBinding BorderBrush}"
    BorderThickness="{TemplateBinding BorderThickness}">
    <Grid Margin="{TemplateBinding Padding}">
      <vki:DigitalNumberPresenter
        Value="{TemplateBinding Value}"
        DigitalFont="{TemplateBinding DigitalFont}"
        SegmentFilter="InactiveOnly"
        Fill="{TemplateBinding Inactive}"
        Stroke="{TemplateBinding InactiveStroke}"
        .../>
      <vki:DigitalNumberPresenter
        Value="{TemplateBinding Value}"
        DigitalFont="{TemplateBinding DigitalFont}"
        Fill="{TemplateBinding Active}"
        Stroke="{TemplateBinding ActiveStroke}"
        .../>
    </Grid>
  </Border>
</ControlTemplate>
```

XAML



Digital Indicator

Digital Indicator Presenter

- SegmentFilter = **InactiveOnly**

Digital Indicator Presenter

- SegmentFilter = **ActiveOnly**

Drawing Context를 이용한 렌더링



```
public abstract class DigitalIndicatorPresenter : FrameworkElement, IDigitalIndicatorPresenter
{
    ...
    private readonly DashStyle dashStyle = new DashStyle();
    private readonly PartDrawingContext partDrawingContext = new PartDrawingContext();
    private readonly Pen strokePen = new Pen();

    protected override void OnRender(DrawingContext drawingContext)
    {
        strokePen.DashStyle = StrokeDashArray == null && StrokeDashOffset == 0.0 ? null : dashStyle;
        var strokeThickness = strokePen.Thickness;
        var geometry = new StreamGeometry();
        using (var geometryContext = geometry.Open())
        {
            partDrawingContext.Renderer = geometryContext;
            foreach (var part in OnCreateParts())
                partDrawingContext.DrawPart(part);
        }
        geometry.Freeze();
        drawingContext.DrawGeometry(Fill, strokeThickness > 0 && strokePen.Brush != null ? strokePen : null, geometry);
    }

    protected abstract IEnumerable<Part> OnCreateParts();
    ...
}
```



DigitalIndicator 클래스



API 레퍼런스 / 네임스페이스 / VagabondK.Indicators.Avalonia / 클래스 / DigitalIndicator / 필드 / 속성

DigitalIndicator 클래스

값을 여러 세그먼트로 구성된 디지털 문자들로 표시하는 인디케이터입니다.

상속

```
↳ object
  ↳ AvaloniaObject
    ↳ Animatable
      ↳ StyledElement
        ↳ Visual
          ↳ Layoutable
            ↳ Interactive
              ↳ InputElement
                ↳ Control
                  ↳ TemplatedControl
                    ↳ DefaultTemplatedControl
                      ↳ DigitalIndicator
                        ↳ DigitalNumber
                        ↳ DigitalText
```

구현

INotifyPropertyChanged
ISupportInitialize
IDigitalIndicator
IIndicator

네임스페이스: VagabondK.Indicators.Avalonia

어셈블리: VagabondK.Indicators.Avalonia.dll

속성

이름	설명
Active	활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
ActiveStroke	활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
Inactive	비활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
InactiveStroke	비활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
StrokeDashArray	윤곽선의 대시 및 간격 컬렉션을 가져오거나 설정합니다.
StrokeDashOffset	대시 시퀀스와 윤곽선 시작 위치 사이의 간격을 가져오거나 설정합니다.
StrokeJoin	윤곽선의 꼭짓점에 사용되는 이음새의 형식을 가져오거나 설정합니다.
StrokeLineCap	윤곽선의 끝 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeThickness	세그먼트 윤곽선 두께를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalIndicatorPresenter 클래스



API 레퍼런스 / 네임스페이스 / VagabondK.Indicators.Avalonia / 클래스 / DigitalIndicatorPresenter / 생성자

DigitalIndicatorPresenter 클래스

표시할 세그먼트에 대한 필터에 따라 디지털 문자들을 표시합니다.

상속

```
↳ object
  ↳ AvaloniaObject
    ↳ Animatable
      ↳ StyledElement
        ↳ Visual
          ↳ Layoutable
            ↳ Interactive
              ↳ InputElement
                ↳ Control
                  ↳ DigitalIndicatorPresenter
                    ↳ DigitalNumberPresenter
                    ↳ DigitalTextPresenter
```

구현

INotifyPropertyChanged
ISupportInitialize
IDigitalIndicatorPresenter
IDigitalIndicator
IIndicator

네임스페이스: VagabondK.Indicators.Avalonia

어셈블리: VagabondK.Indicators.Avalonia.dll

속성

이름	설명
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
Fill	세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
SegmentFilter	표시할 세그먼트에 대한 필터를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
Stroke	세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
StrokeDashArray	윤곽선의 대시 및 간격 컬렉션을 가져오거나 설정합니다.
StrokeDashOffset	대시 시퀀스와 윤곽선 시작 위치 사이의 간격을 가져오거나 설정합니다.
StrokeJoin	윤곽선의 꼭짓점에 사용되는 이음새의 형식을 가져오거나 설정합니다.
StrokeLineCap	윤곽선의 끝 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeThickness	세그먼트 윤곽선 두께를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalNumber 스타일 정의



```
<Styles xmlns="https://github.com/avaloniaui"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:vki="http://schemas.vagabond-k.com/avaloniaui/indicators"
  xmlns:vkid="using:VagabondK.Indicators.DigitalFonts">
  <Design.PreviewWith>
    <vki:DigitalNumber />
  </Design.PreviewWith>
  <Style Selector="vki|DigitalNumber">
    <Setter Property="DigitalFont">
      <Setter.Value>
        <vkid:SevenSegmentFont/>
      </Setter.Value>
    </Setter>
    <Setter Property="ClipToBounds" Value="False"/>
    <Setter Property="Template">
      <ControlTemplate>
        <Border Background="{TemplateBinding Background}"
          BorderBrush="{TemplateBinding BorderBrush}"
          BorderThickness="{TemplateBinding BorderThickness}">
          <Grid Margin="{TemplateBinding Padding}">
            <vki:DigitalNumberPresenter
              Value="{TemplateBinding Value}"
              DigitalFont="{TemplateBinding DigitalFont}"
              SegmentFilter="InactiveOnly"
              Fill="{TemplateBinding Inactive}"
              Stroke="{TemplateBinding InactiveStroke}"
              ...
```

AXAML



Digital Indicator

Digital Indicator Presenter

- SegmentFilter = InactiveOnly

Digital Indicator Presenter

- SegmentFilter = ActiveOnly

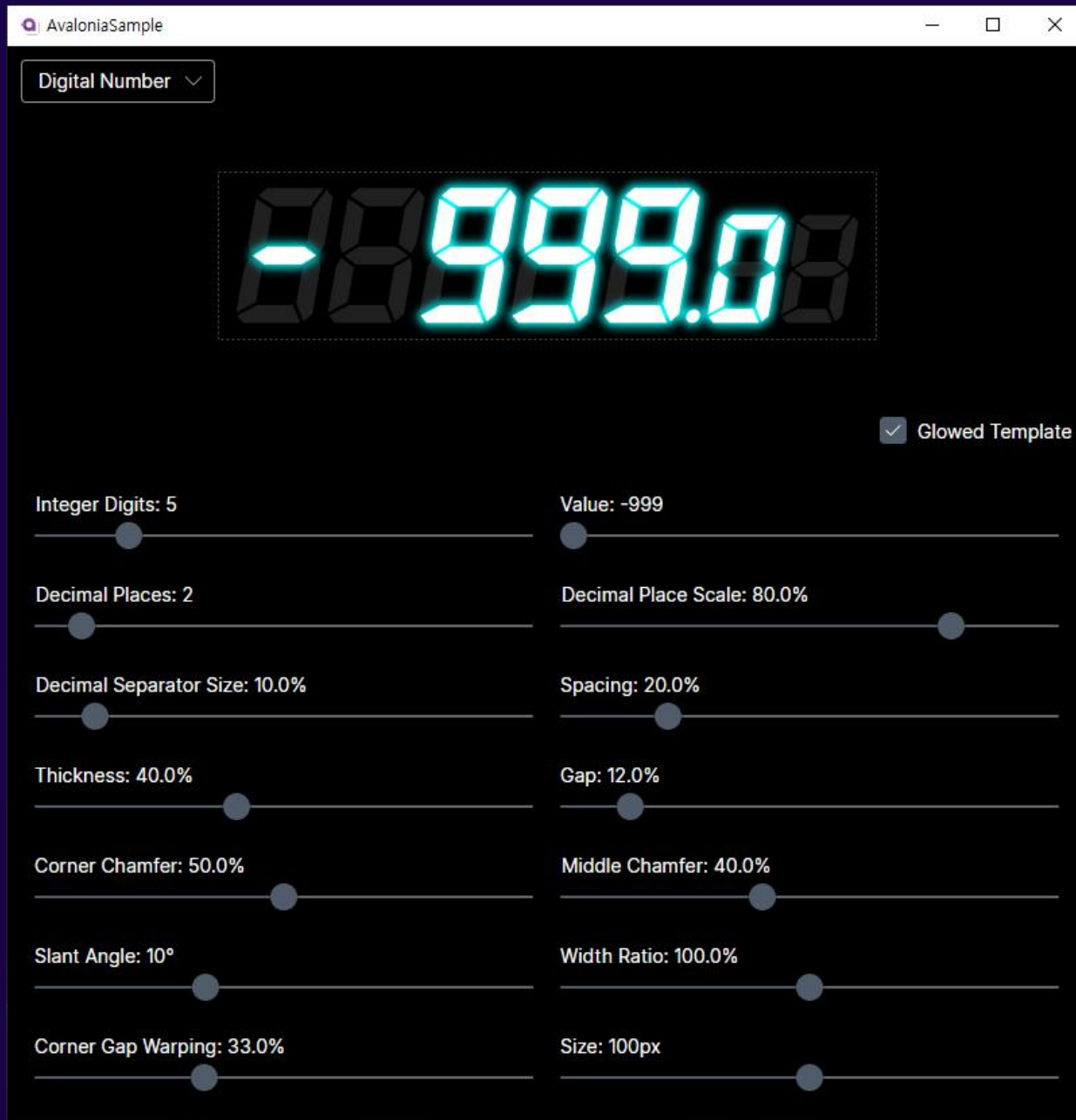
Drawing Context를 이용한 렌더링



```
public abstract class DigitalIndicatorPresenter : Control, IDigitalIndicatorPresenter
{
    ...
    private readonly PartDrawingContext partDrawingContext = new PartDrawingContext();
    private readonly Pen strokePen = new Pen() { DashStyle = new DashStyle() };

    public override void Render(DrawingContext context)
    {
        var strokeThickness = strokePen.Thickness;
        var geometry = new StreamGeometry();
        using (var geometryContext = geometry.Open())
        {
            partDrawingContext.Renderer = geometryContext;
            foreach (var part in OnCreateParts())
                partDrawingContext.DrawPart(part);
        }
        context.DrawGeometry(Fill, strokeThickness > 0 && strokePen.Brush != null ? strokePen : null, geometry);
    }

    protected abstract IEnumerable<Part> OnCreateParts();
    ...
}
```

DigitalIndicator 클래스



[API 레퍼런스](#) / [네임스페이스](#) / [VagabondK.Indicators.Maui](#) / [클래스](#) / [DigitalIndicator](#) / [필드](#) / [속성](#)

DigitalIndicator 클래스

값을 여러 세그먼트로 구성된 디지털 문자들로 표시하는 인디케이터입니다.

상속

```
↳ object
  ↳ BindableObject
    ↳ Element
      ↳ NavigableElement
        ↳ VisualElement
          ↳ View
            ↳ Layout
              ↳ TemplatedView
                ↳ DigitalIndicator
                  ↳ DigitalNumber
                  ↳ DigitalText
```

구현

[INotifyPropertyChanged](#)
[IDigitalIndicator](#)
[Indicator](#)

네임스페이스: [VagabondK.Indicators.Maui](#)
어셈블리: [VagabondK.Indicators.Maui.dll](#)

속성

이름	설명
Active	활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
ActiveStroke	활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
Inactive	비활성 세그먼트 영역을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
InactiveStroke	비활성 세그먼트 윤곽선을 그릴 때 사용할 브러시를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
StrokeDashArray	윤곽선의 대시 및 간격 컬렉션을 가져오거나 설정합니다.
StrokeDashOffset	대시 시퀀스와 윤곽선 시작 위치 사이의 간격을 가져오거나 설정합니다.
StrokeLineCap	윤곽선의 끝 부분에 사용할 도형 형식을 가져오거나 설정합니다.
StrokeLineJoin	윤곽선의 꼭짓점에 사용되는 이음새의 형식을 가져오거나 설정합니다.
StrokeMiterLimit	마이터 길이의 비율 제한을 가져오거나 설정합니다.
StrokeThickness	세그먼트 윤곽선 두께를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalIndicatorPresenter 클래스



API 레퍼런스 / 네임스페이스 / VagabondK.Indicators.Maui / 클래스 / DigitalIndicatorPresenter / 필드 / 속

DigitalIndicatorPresenter 클래스

표시할 세그먼트에 대한 필터에 따라 디지털 문자들을 표시합니다.

상속

```
↳ object
  ↳ BindableObject
    ↳ Element
      ↳ NavigableElement
        ↳ VisualElement
          ↳ View
            ↳ Shape
              ↳ DigitalIndicatorPresenter
                ↳ DigitalNumberPresenter
                ↳ DigitalTextPresenter
```

구현

INotifyPropertyChanged
IDigitalIndicatorPresenter
IDigitalIndicator
IIndicator

네임스페이스: VagabondK.Indicators.Maui

어셈블리: VagabondK.Indicators.Maui.dll

이름	설명
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
SegmentFilter	표시할 세그먼트에 대한 필터를 가져오거나 설정합니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

Shape 클래스 (Microsoft.Maui.Co	+
https://learn.microsoft.com/ko-kr/dotn...	
AspectProperty	에 대한 바인딩 가능한 속성입니다 Aspect.
FillProperty	에 대한 바인딩 가능한 속성입니다 Fill.
propertyMapper	뷰의 내부 PropertyMapper를 나타냅니다. (다음에서 상속됨 View)
StrokeDashArrayProperty	에 대한 바인딩 가능한 속성입니다 StrokeDashArray.
StrokeDashOffsetProperty	에 대한 바인딩 가능한 속성입니다 StrokeDashOffset.
StrokeLineCapProperty	에 대한 바인딩 가능한 속성입니다 StrokeLineCap.
StrokeLineJoinProperty	에 대한 바인딩 가능한 속성입니다 StrokeLineJoin.

DigitalNumber 템플릿 정의



```
<?xml version="1.0" encoding="utf-8" ?>
<vki:DigitalIndicator
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:vki="clr-namespace:VagabondK.Indicators.Maui"
  x:Class="VagabondK.Indicators.Maui.DigitalNumber">
  <TemplatedView.ControlTemplate>
    <ControlTemplate>
      <Grid Background="{TemplateBinding Background}" Margin="{TemplateBinding Padding}">
        <vki:DigitalNumberPresenter
          Value="{TemplateBinding Value}"
          DigitalFont="{TemplateBinding DigitalFont}"
          SegmentFilter="InactiveOnly"
          Fill="{TemplateBinding Inactive}"
          Stroke="{TemplateBinding InactiveStroke}"
          ...
          Aspect="{TemplateBinding Aspect}"/>
        <vki:DigitalNumberPresenter
          Value="{TemplateBinding Value}"
          DigitalFont="{TemplateBinding DigitalFont}"
          Fill="{TemplateBinding Active}"
          Stroke="{TemplateBinding ActiveStroke}"
          ...
          Aspect="{TemplateBinding Aspect}"/>
      </Grid>
    </ControlTemplate>
  </TemplatedView.ControlTemplate>
</vki:DigitalIndicator>
```

XAML



Digital Indicator

Digital Indicator Presenter

- SegmentFilter = InactiveOnly

Digital Indicator Presenter

- SegmentFilter = ActiveOnly

Drawing Context를 이용한 렌더링

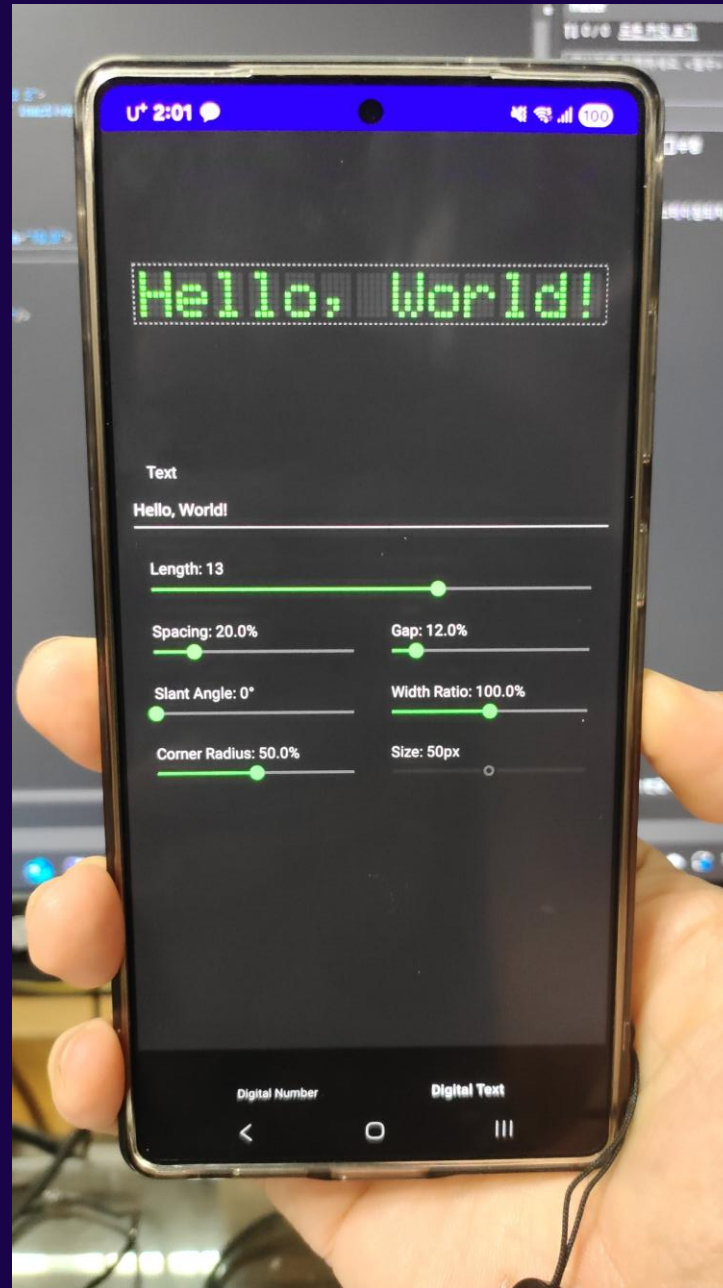


```
public abstract class DigitalIndicatorPresenter : Shape, IView, IShape, IDigitalIndicatorPresenter
{
    ...
    private PathF path = new();
    private readonly PartDrawingContext partDrawingContext = new PartDrawingContext();

    protected abstract IEnumerable<Part> OnCreateParts();

    public override PathF GetPath() => path;

    PathF IShape.PathForBounds(Rect viewBounds)
    {
        path = new PathF();
        partDrawingContext.Renderer = path;
        foreach (var part in OnCreateParts())
            partDrawingContext.DrawPart(part);
        ...
        return path;
    }
    ...
}
```



DigitalIndicator 클래스



[API 레퍼런스](#) / [네임스페이스](#) / [VagabondK.Indicators.Razor](#) / 클래스

DigitalIndicator 클래스

값을 여러 세그먼트로 구성된 디지털 문자들로 표시하는 인디케이터

상속

↳ object
↳ ComponentBase
↳ DigitalIndicator
↳ DigitalNumber
↳ DigitalText

구현

IDigitalIndicator

IIndicator

네임스페이스: [VagabondK.Indicators.Razor](#)

어셈블리: [VagabondK.Indicators.Razor.dll](#)

속성

이름	설명
ActiveClass	활성 세그먼트 영역 svg 태그의 class 특성에 적용할 내용을 가져오거나 설정합니다. 해당 class 특성에는 항상 자동으로 vki-digital-presenter 클래스가 추가됩니다.
ActiveStyle	활성 세그먼트 영역 svg 태그의 style 특성에 적용할 내용을 가져오거나 설정합니다.
Class	인디케이터 컴포넌트에서 생성할 svg 태그의 class 특성에 적용할 내용을 가져오거나 설정합니다.
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
InactiveClass	비활성 세그먼트 영역 svg 태그의 class 특성에 적용할 내용을 가져오거나 설정합니다. 해당 class 특성에는 항상 자동으로 vki-digital-presenter 클래스가 추가됩니다.
InactiveStyle	비활성 세그먼트 영역 svg 태그의 style 특성에 적용할 내용을 가져오거나 설정합니다.
RenderAllInactive	활성 세그먼트 뒤에 비활성 세그먼트도 함께 렌더링 여부를 가져오거나 설정합니다.
Size	SVG의 크기를 가져옵니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
Style	인디케이터 컴포넌트에서 생성할 svg 태그의 style 특성에 적용할 내용을 가져오거나 설정합니다.
SvgDefs	svg 태그의 defs 태그 내용을 가져오거나 설정합니다.
UseSharedSegments	공유 세그먼트를 사용할지 여부를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalIndicatorPresenter 클래스



[API 레퍼런스](#) / [네임스페이스](#) / [VagabondK.Indicators.Razor](#) / [클래스](#) / [DigitalIndicatorPresenter](#)

DigitalIndicatorPresenter 클래스

표시할 세그먼트에 대한 필터에 따라 디지털 문자들을 표시합니다.

상속

↳ object
↳ ComponentBase
↳ DigitalIndicatorPresenter
↳ [DigitalNumberPresenter](#)
↳ [DigitalTextPresenter](#)

구현

[IDigitalIndicatorPresenter](#)
[IDigitalIndicator](#)
[IIndicator](#)
네임스페이스: [VagabondK.Indicators.Razor](#)
어셈블리: [VagabondK.Indicators.Razor.dll](#)

속성

이름	설명
Class	프리젠퍼 컴포넌트에서 생성할 svg 태그의 class 특성에 적용할 내용을 가져오거나 설정합니다. 해당 class 특성에는 항상 자동으로 vki-digital-presenter 클래스가 추가됩니다.
DigitalFont	디지털 문자 양식을 가져오거나 설정합니다.
SegmentFilter	표시할 세그먼트에 대한 필터를 가져오거나 설정합니다.
Size	SVG의 크기를 가져옵니다.
Spacing	디지털 문자의 가로 크기 대비 문자간 거리 비율을 가져오거나 설정합니다.
Style	프리젠퍼 컴포넌트에서 생성할 svg 태그의 style 특성에 적용할 내용을 가져오거나 설정합니다.
SvgDefs	svg 태그의 defs 태그 내용을 가져오거나 설정합니다.
UseSharedSegments	공유 세그먼트를 사용할지 여부를 가져오거나 설정합니다.
Value	화면에 표시할 값을 가져오거나 설정합니다.

DigitalNumber 컴포넌트



Razor

```
@inherits DigitalIndicator
<svg class="vki-digital-number @Class" style="@Style" width="@Size.Width" height="@Size.Height" viewBox="0 0 @Size.Width @Size.Height"
    xmlns="http://www.w3.org/2000/svg">
    <defs>
        @SvgDefs
    </defs>
    <g style="color:#808080;fill-opacity:0.25;">
        <DigitalNumberPresenter
            Class="@InactiveClass"
            Style="@InactiveStyle"
            Value="@Value"
            Spacing="@Spacing"
            DigitalFont="@DigitalFont"
            ...
            UseSharedSegments="@UseSharedSegments"
            SegmentFilter="@((RenderAllInactive ? DigitalFonts.DigitalSegmentFilter.All : DigitalFonts.DigitalSegmentFilter.InactiveOnly))" />
    </g>
    <g>
        <DigitalNumberPresenter
            Class="@ActiveClass"
            Style="@ActiveStyle"
            Value="@Value"
            ...
            UseSharedSegments="true"
            SegmentFilter="@DigitalFonts.DigitalSegmentFilter.ActiveOnly" />
    </g>
</svg>
```

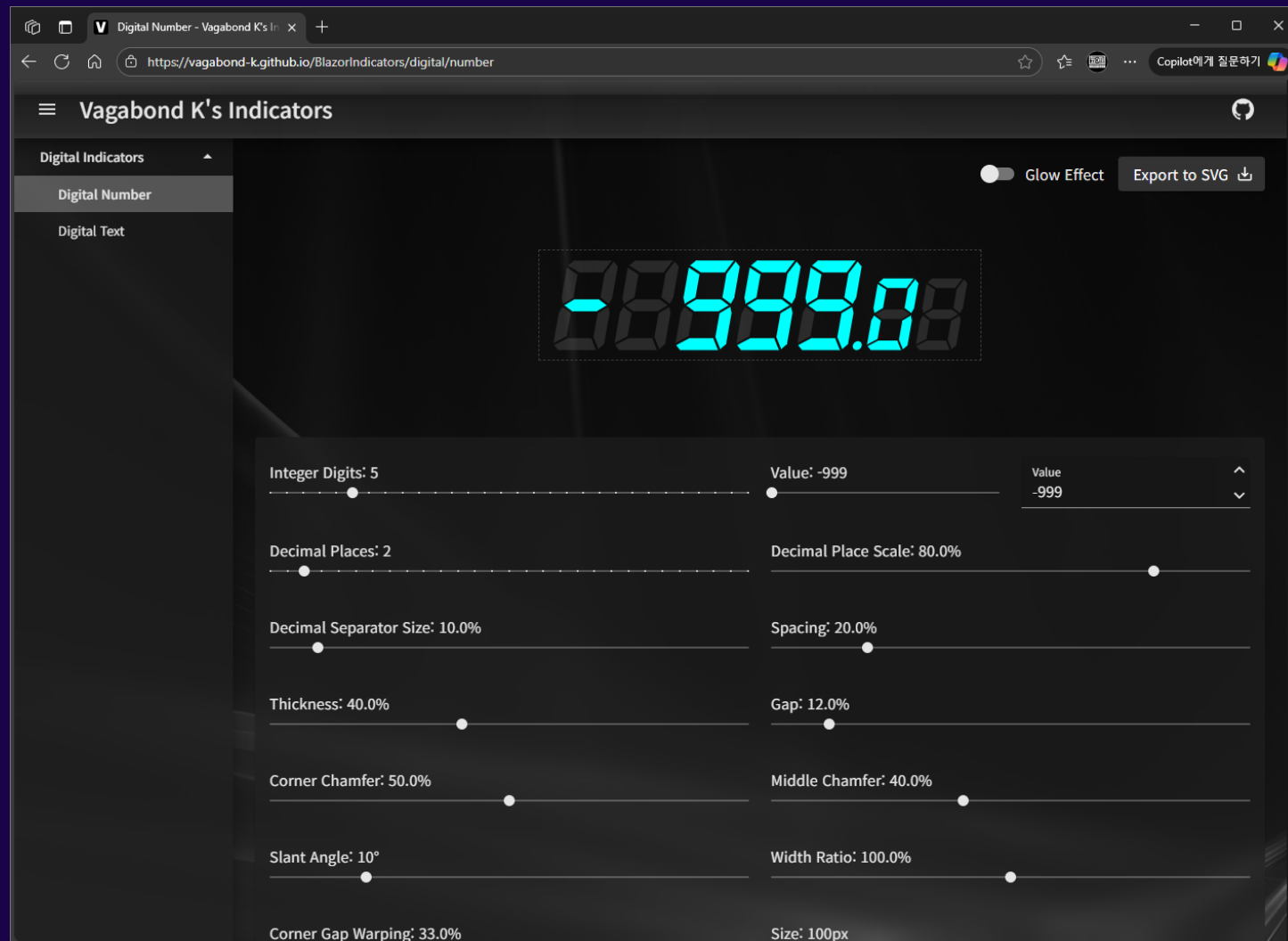
DigitalNumberPresenter 컴포넌트



Razor

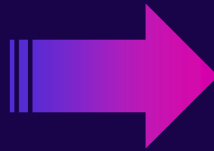
```
<svg class="vki-digital-presenter @Class" style="fill:currentColor;@Style" width="@Size.Width" height="@Size.Height" viewBox="0 0 @Size.Width @Size.Height"
xmlns="http://www.w3.org/2000/svg">
  @if (!UseSharedSegments)
  {
    <defs>
      @for (int i = 0; i < segmentPaths.Count; i++)
      {
        <path id="@ (segmentIdPrefix + i)" d="@segmentPaths[i]" />
      }
      @SvgDefs
    </defs>
  }
  <g class="@defaultClassName">
    @foreach (var item in segmentDrawings)
    {
      <use href="#@item.Value"
        transform="matrix(@item.Transform.M11 @item.Transform.M12 @item.Transform.M21 @item.Transform.M22 @item.Transform.M31 @item.Transform.M32)" />
    }
    @foreach (var item in customDrawings)
    {
      <path d="@item.Value"
        transform="matrix(@item.Transform.M11 @item.Transform.M12 @item.Transform.M21 @item.Transform.M22 @item.Transform.M31 @item.Transform.M32)" />
    }
  </g>
</svg>
```

<https://vagabond-k.github.io/BlazorIndicators>



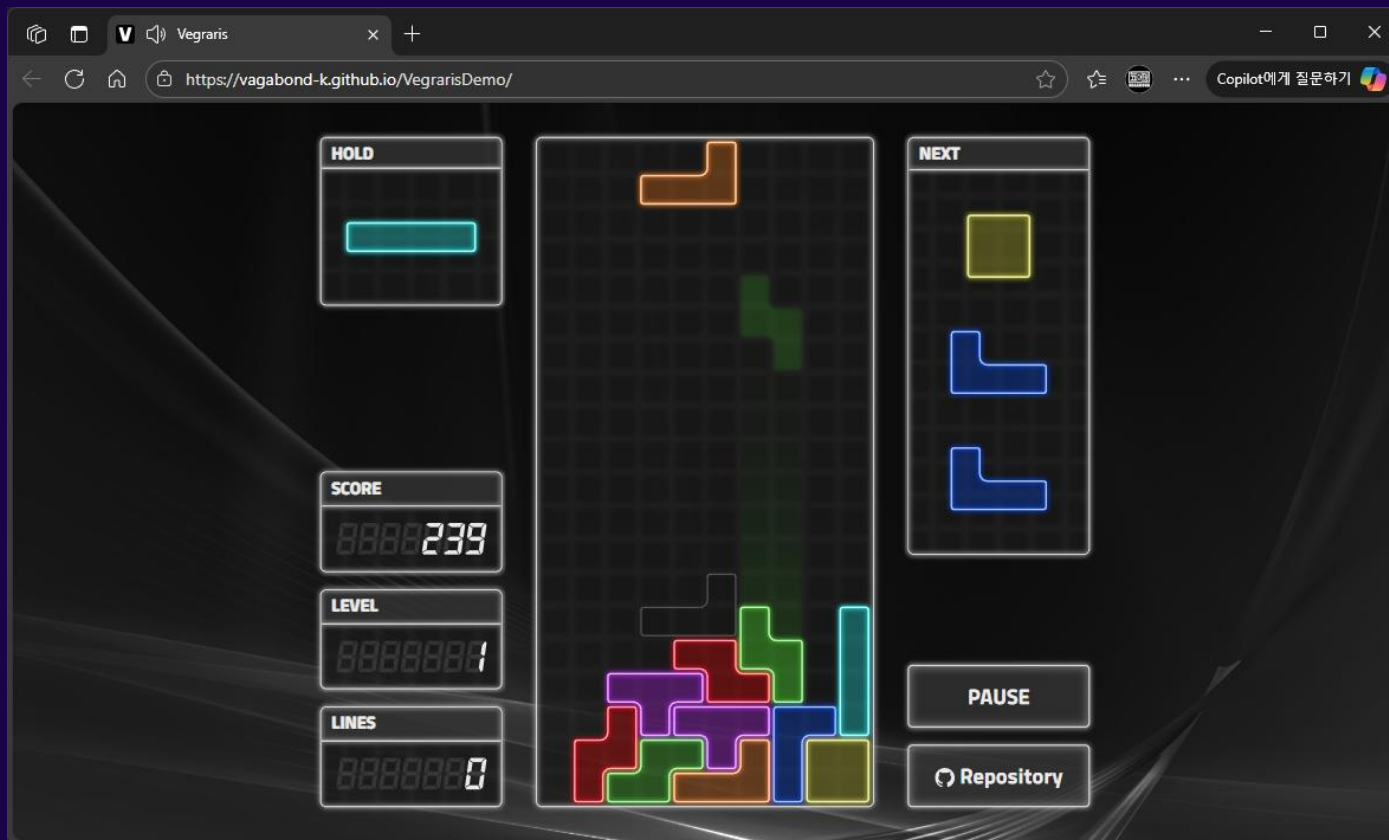
04

사용 예시 및 버그 개선



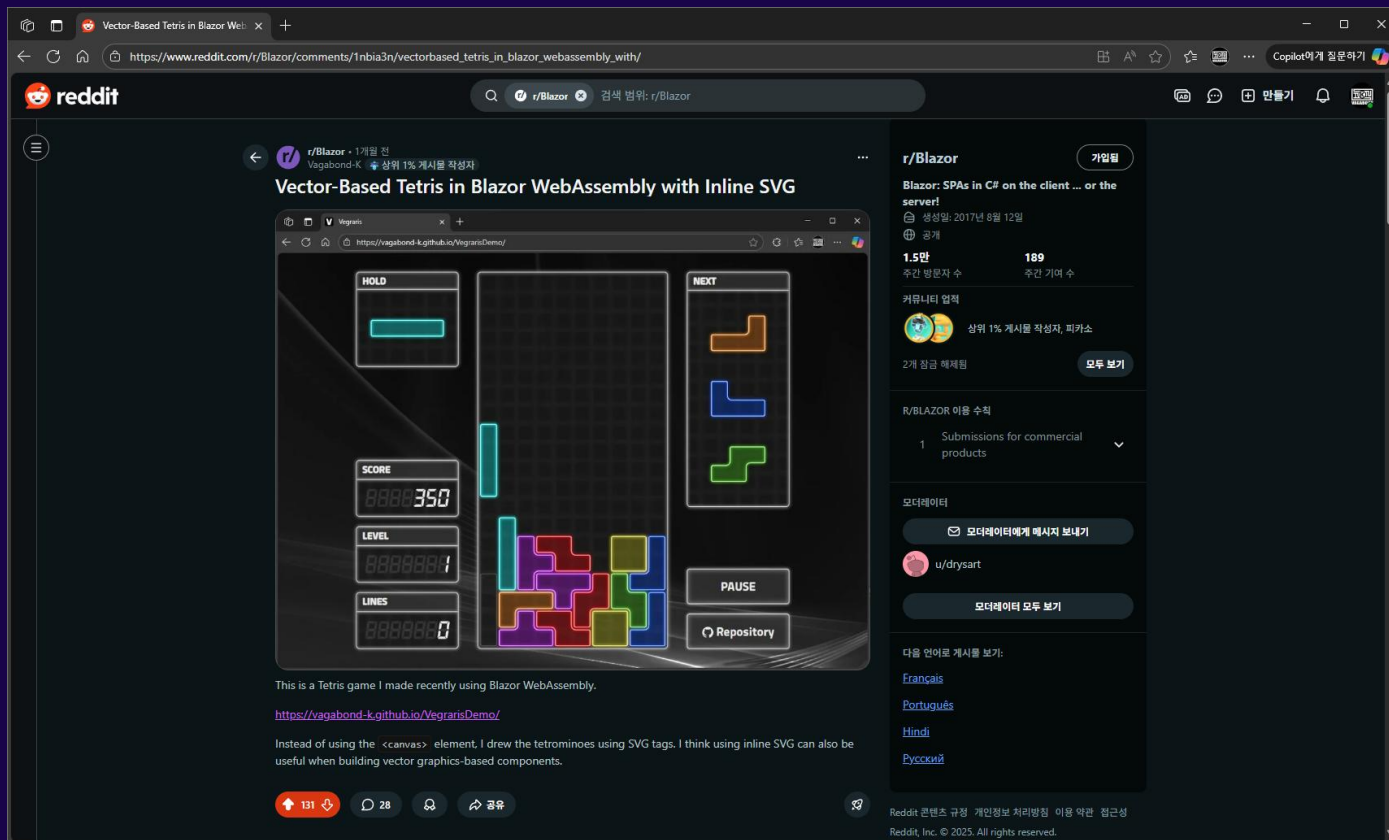
VEGRARIS – Vector Graphics 기반 TETRIS

<https://vagabond-k.github.io/VegrarisDemo>



Reddit 공유

https://www.reddit.com/r/Blazor/comments/1nbia3n/vectorbased_tetris_in_blazor_webassembly_with



문화권별 숫자 형식에 의한 오류 피드백



pm303 · 1개월 전

Looks great!

But doesn't work for me. I'm pretty sure it's because my local settings are different from yours.

Error: <path> attribute d: Expected number, "...38850520242085 0L 23,91371616471...".

Error: <path> attribute d: Expected number, "...38850520242085 0L 23,91371616471...".

Error: <path> attribute d: Expected number, "...38850520242085 0L 23,91371616471...".

⊖ ↑ 6 ↓ 💬 답글 달기 🏆 어워드 ➦ 공유 ...



KrisStrube · 1개월 전

That is true when working with numbers in the DOM, you should either use Invariant Culture for the entire project which can be setup in the csproj or you should explicitly use `.ToString(CultureInfo.InvariantCulture)` whenever you insert a floating point number in the DOM.

⊖ 🔼 5 🔽 💬 답글 달기 🏆 어워드 ➦ 공유 ...



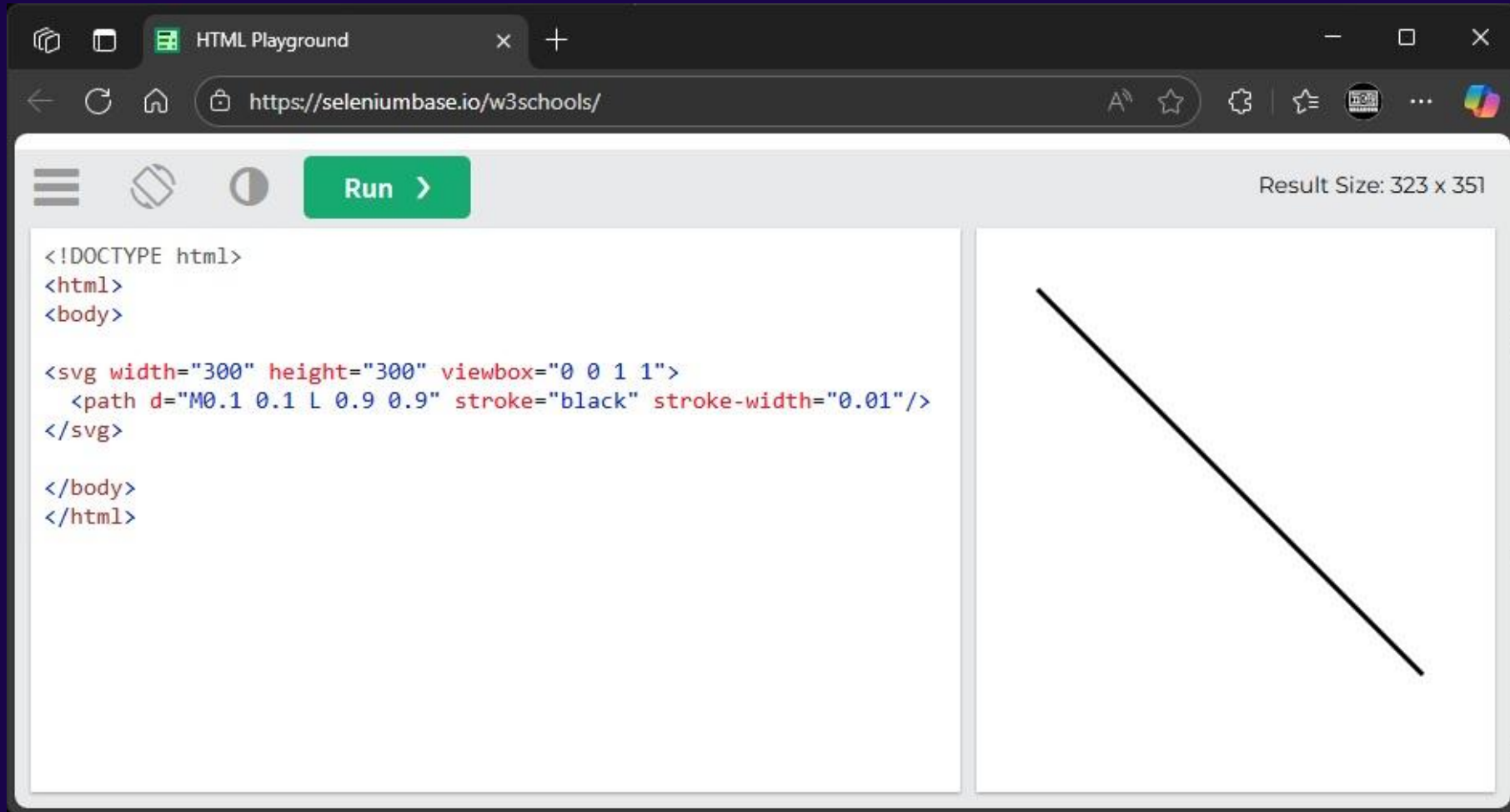
Vagabond-K **원글 작성자** · 1개월 전

🏆 상위 1% 게시물 작성자

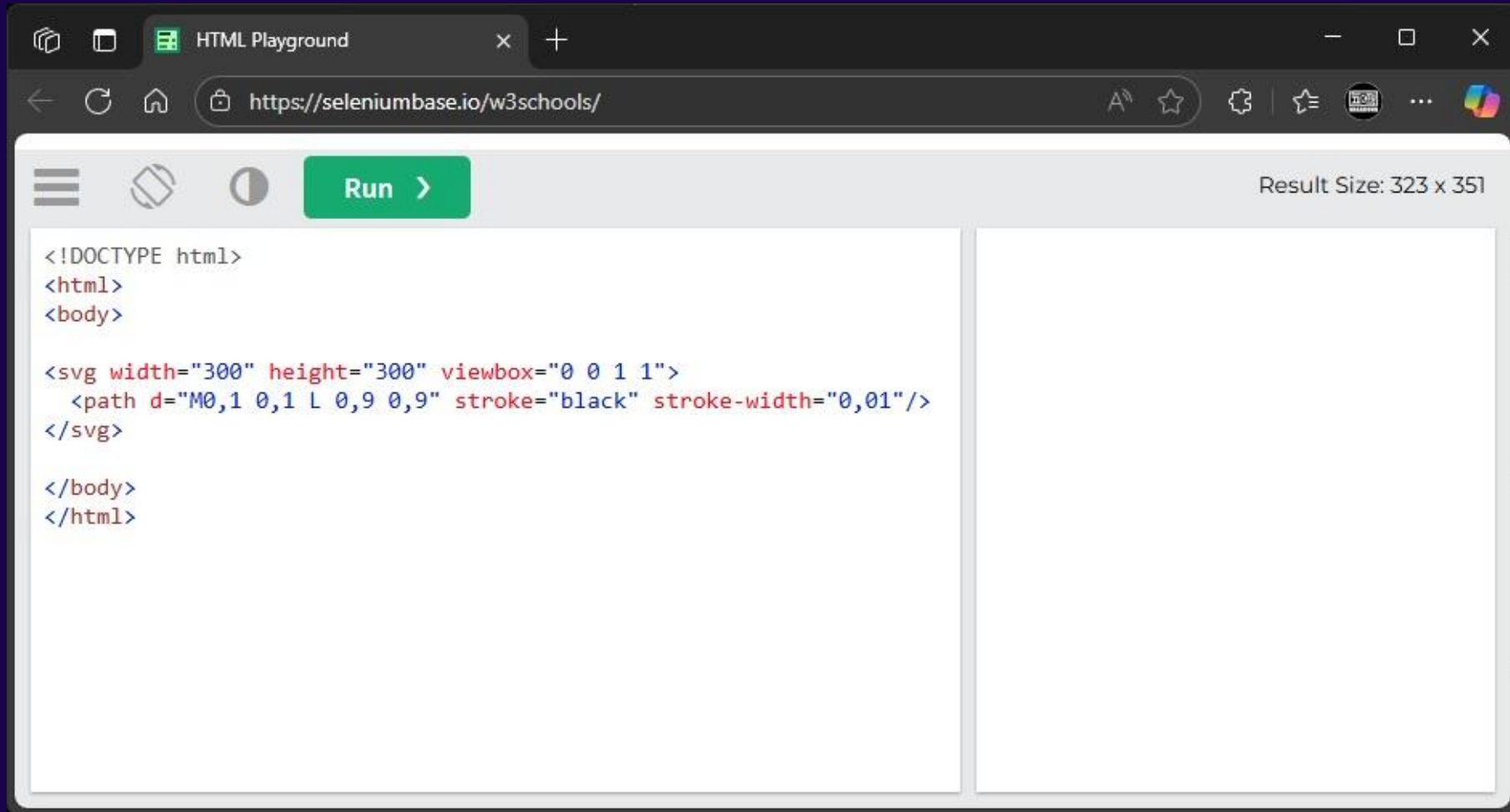
Thanks for the great tip! I've applied the change.

🔼 2 🔽 💬 답글 달기 🏆 어워드 ➦ 공유 ⌚ 298 ...

문화권별 숫자 형식에 의한 오류 피드백



문화권별 숫자 형식에 의한 오류 피드백



CultureInfo.InvariantCulture

The screenshot shows the Microsoft Learn website for the `CultureInfo.InvariantCulture` property in Korean. The page is titled "CultureInfo.InvariantCulture 속성" (CultureInfo.InvariantCulture Property). The left sidebar shows a tree view of the `CultureInfo` class members, with `InvariantCulture` selected. The main content area includes the following sections:

- 버전** (Version): A dropdown menu showing ".NET 8".
- Learn / .NET / API 브라우저 /** (Breadcrumbs)
- Focus mode** (Toggle)
- C#** (Language selector)
- 이 문서의 내용** (Table of Contents):
 - 정의 (Definition)
 - 설명 (Description)
 - 적용 대상 (Applies to)
 - 추가 정보 (Additional information)
- 이 페이지가 도움이 되었나요?** (Feedback): "Yes" and "No" buttons.

The main content area contains the following information:

- 정의** (Definition):
 - 네임스페이스: `System.Globalization`
 - 어셈블리: `System.Runtime.dll`
 - Source: [CultureInfo.cs](#)
- 문화권 독립(고정)적인 CultureInfo 개체를 가져옵니다.** (Retrieves a culture-independent (fixed) CultureInfo object.)
- C#** (Code example):

```
public static System.Globalization.CultureInfo InvariantCulture { get; }
```
- 속성 값** (Property value):
 - `CultureInfo`
 - 문화권 독립(고정)적인 개체입니다. (A culture-independent object.)

Blazor 기반 Drawing Context 수정



C#

```
public class SvgPathDrawingContext : PartDrawingContext<StringBuilder>
{
    private const char space = ' ';
    protected override void OnBeginPath(in Point startPoint) => Renderer.Append('M').AppendPoint(startPoint);
    protected override void OnDrawLine(in Point endPoint) => Renderer.Append('L').AppendPoint(endPoint);
    protected override void OnDrawCubicBezier(in Point controlPoint1, in Point controlPoint2, in Point endPoint)
        => Renderer.Append('C').AppendPoint(controlPoint1).AppendPoint(controlPoint2).AppendPoint(endPoint);
    protected override void OnDrawQuadraticBezier(in Point controlPoint, in Point endPoint)
        => Renderer.Append('Q').AppendPoint(controlPoint).AppendPoint(endPoint);
    protected override void OnClosePath() => Renderer.Append('Z');
}

static class StringBuilderExtensions
{
    // public static StringBuilder AppendPoint(this StringBuilder stringBuilder, in Point point)
    //     => stringBuilder.Append(' ').Append(point.X).Append(' ').Append(point.Y);

    public static StringBuilder AppendPoint(this StringBuilder stringBuilder, in Point point)
        => stringBuilder.Append(' ').AppendFormat(CultureInfo.InvariantCulture, "{0}", point.X)
            .Append(' ').AppendFormat(CultureInfo.InvariantCulture, "{0}", point.Y);
}
```

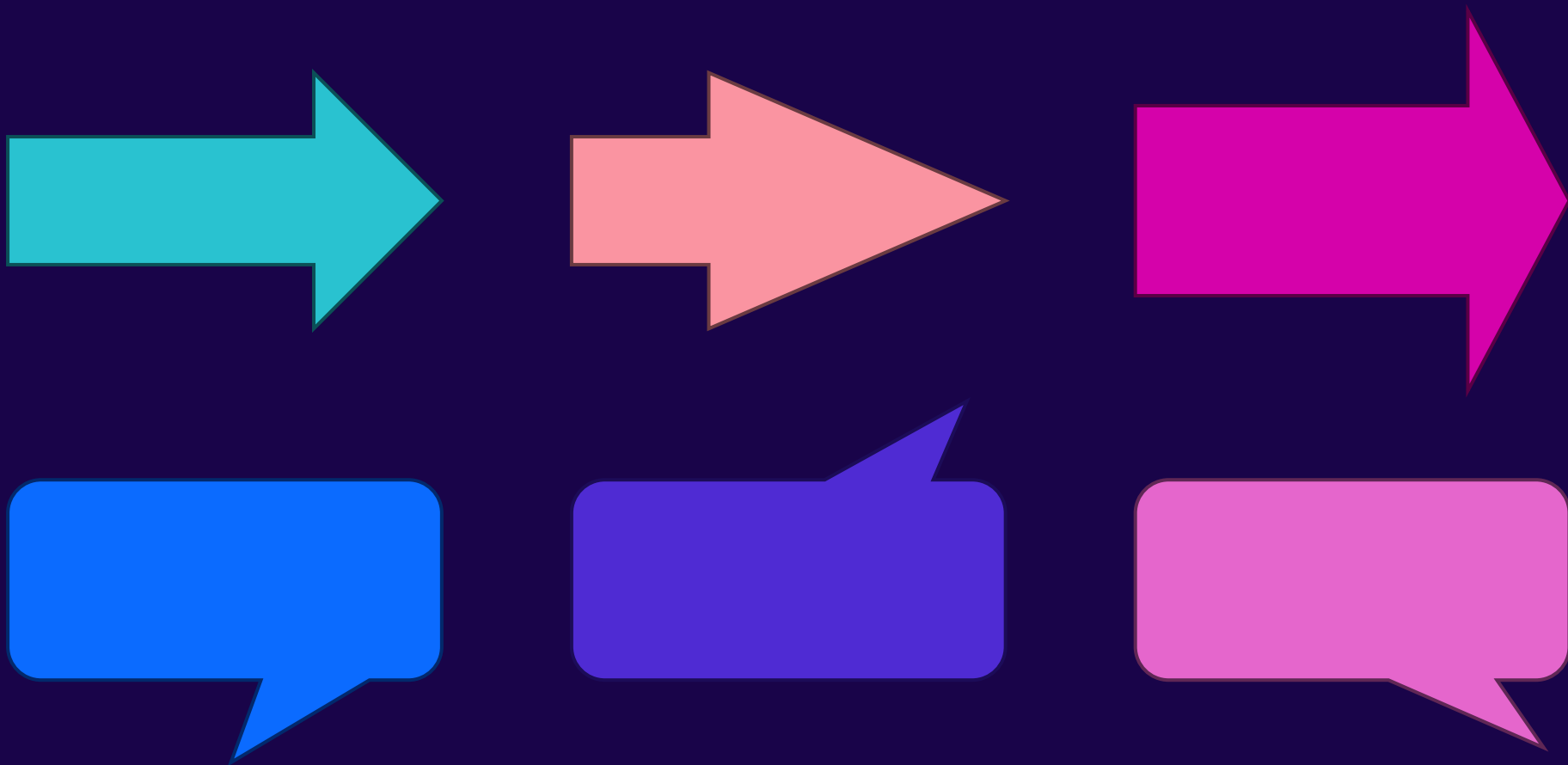
05

향후 개발 방향

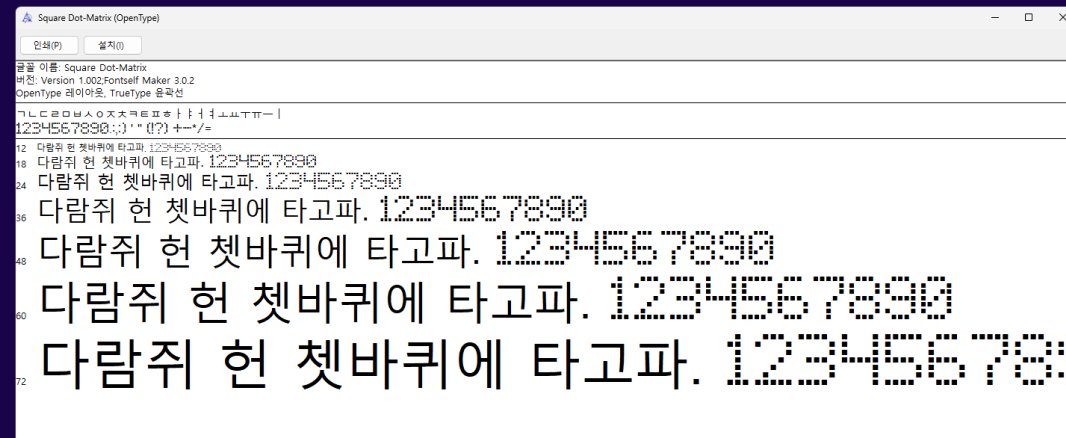
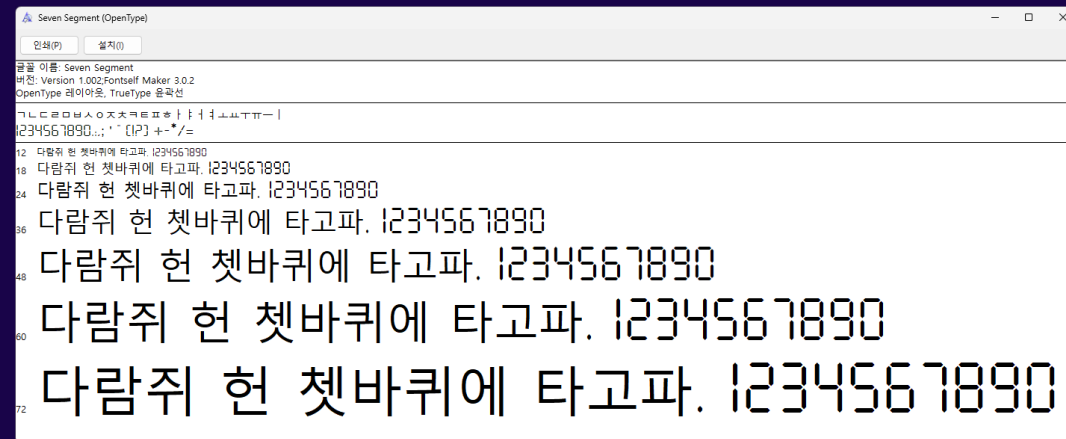
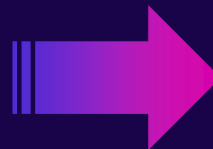
아날로그 인디케이터



파라미터 기반 도형 라이브러리



폰트 파일 자동 생성 기능



소스코드 및 참고자료

- Source code

- <https://github.com/Vagabond-K/VagabondK.Indicators>

- API reference documents

- <https://vagabond-k.github.io/docs/api/VagabondK.Indicators.html>

- Blazor showcase

- <https://vagabond-k.github.io/BlazorIndicators>

- Blog article

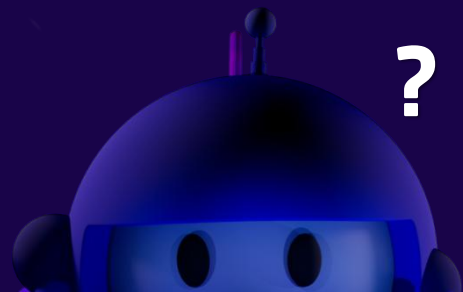
- <https://blog.naver.com/vagabond-k/223481454735>

- SVG path tag

- https://developer.mozilla.org/ko/docs/Web/SVG/Tutorials/SVG_from_scratch/Paths

Thank you

But...



발표자 소개

- 경남 창원외의 어느 중소기업에서 20년 가까이 개발자로 근무 중
- 에너지 관리 시스템 개발 및 에너지 기술 연구 과제 수행
- 연구기관 SW 개발 용역 수행 (한국전기연구원, 한국전자기술연구원 등)
- 닷넷데브의 .NET Conf 2025 × Seoul 연사 참여
- 개인 활동: Blog, GitHub, LinkedIn, YouTube
- 닉네임: Vagabond K



www.vagabond-k.com

Thank you

